



Griffith College

A Hybrid SecureBERT-BiLSTM-Attention Model for Cyber Threat Intelligence Classification

Saikiran Achuta

Student Number: 3196603

Programme: MSc in Computing Science

Submitted in partial fulfilment of the requirements for the degree of
of Science in Computing science

Griffith College Dublin

Supervisor: Abubakr Siddig

September 2026

Disclaimer

I hereby declare that this work is original and that I have not submitted it for assessment for an academic purpose anywhere else or for the award of any academic qualification at this or any other academic institution except as a part of the programme of study leading to the Degree of Master of Science in Computing Science at Griffith College Dublin.

Saikiran Achuta

08-09-2026

Acknowledgement

I would like to express my gratitude to everyone who had contributed to the completion of this dissertation project “A Hybrid SecureBERT-BiLSTM-Attention Model for Cyber Threat Intelligence Classification”.

I would like to thank my project supervisor for his guidance, feedback and support throughout the research and development process, especially the feedback provided on the Week 4 submission which guided the direction of the final model development, evaluation and robustness testing presented in this report.

I would like to thank the Computing Science faculty and staff at Griffith College for providing me with a space to do applied research and independent problem solving, as well as the opensource communities of Python, pandas, NumPy, Scikitlearn, NLTK, TensorFlow/Keras, joblib, Matplotlib, and Seaborn. The tools enabled the system to be practically developed and validated end-to-end.

Finally, I would like to thank my family and friends for their patience, encouragement and motivation during this academic process.

Contents

Disclaimer	i
Acknowledgement	ii
List of Figures	vi
List of Tables	vii
Abstract	1
1 Introduction	2
1.1 Overview	2
1.2 Background	2
1.3 Rationale	3
1.4 Problem Statement	3
1.5 Research Aim	4
1.6 Research Objectives	4
1.7 Research Questions	4
1.8 Detailed Technical Justification	5
1.9 Extended Technical Discussion	5
1.10 Research Significance	6
1.11 Research Structure	6
2 Literature Review	8
2.1 Introduction	8
2.2 Foundational NLP Architectures	9
2.3 Domain-Adapted Language Models for Cybersecurity	10
2.4 Hybrid Transformer–Recurrent Architectures for Threat Text Classification	11
2.5 TTP and Entity/Relation Extraction from CTI Text	12
2.6 Large Language Models in CTI and Reliability Concerns	14
2.7 Identified Research Gaps	14
3 Methodology	16
3.1 Overview	16

3.2	Dataset and Data Structure	18
3.3	Data Cleaning and Preprocessing	18
3.4	Construction of the Classification Target	20
3.5	Entity-Aware Exploratory Features	20
3.6	Text-Level Statistical Features	21
3.7	Exploratory Data Analysis	21
3.8	Model Training and Validation	25
3.9	Classical Machine Learning Baselines	25
3.10	BiLSTM Deep Learning Baseline	26
3.11	Transformer and Hybrid Architecture	26
3.12	Evaluation and Error Analysis	27
3.13	Reproducibility and Experimental Workflow	27
3.14	Methodological Considerations and Scope	28
3.15	Streamlit Dashboard Integration	28
3.16	Summary	28
4	System Analysis and Design	30
4.1	Overview of the System Architecture	30
4.2	Preprocessing and Label Construction Module	30
4.3	Feature Engineering Module	31
4.4	Exploratory Analysis Module	32
4.5	Baseline Model Architectures	32
4.6	Proposed Hybrid Architecture	32
4.7	Evaluation Harness	33
4.8	Design Rationale Summary	33
4.9	Summary	34
5	Implementation	35
5.1	Technology Stack	35
5.2	Data Preparation Implementation	35
5.3	Label Construction Implementation	36
5.4	Feature Engineering Implementation	37
5.5	Baseline Model Implementation	37
5.6	Hybrid Model Implementation	37
5.7	Evaluation and Reporting Implementation	38
5.8	Reproducibility Measures	38
5.9	Implementation Challenges	38
5.10	Deployed Streamlit Dashboard Implementation	39
5.11	Summary	39

6	Testing and Evaluation	40
6.1	Baseline Results	40
6.2	Proposed Hybrid Model Results	40
6.3	Overall Model Comparison	41
6.4	Per-Class Performance of the Best Model	42
6.5	Diagnostic Error Analysis	44
6.6	Summary of Findings	44
6.7	Summary	45
7	Conclusion and Future Work	46
7.1	Conclusion	46
7.2	Discussion	46
7.3	Limitations	47
7.4	Practical CTI Triage Deployment Pattern	48
7.5	Reflection on Project Development	48
7.6	Research Contributions	48
7.7	Project Struggles and Solutions	49
7.8	Future Work	49
7.9	Detailed Model Comparison and Design Reasoning	49
7.10	Feature Importance Discussion	50
7.11	Additional Analytical Discussion	50
7.12	Final Validation Against Research Questions	50
7.13	Final Technical Positioning	50
7.14	Summary	50
7.15	Final Word on Technical Depth	51
	Bibliography	52
A	Appendix	54
A.1	Streamlit Dashboard	54

List of Figures

3.1	The methodology workflow for the CTI classification system	17
3.2	Class distribution	19
3.3	category distribution donut	19
3.4	Word Cloud	22
3.5	text_length_outliers	22
3.6	entity_type_frequency	23
3.7	eda_features_by_category	23
3.8	feature_correlation	24
3.9	density_vs_context_scatter	24
4.1	System architecture of the cybersecurity threat classification pipeline . .	31
5.1	data flow diagram of the cybersecurity threat classification pipeline . . .	36
6.1	Models comparison	42
6.2	all_model_metrics_grouped_bar	43
6.3	per class metrics visualized	44
A.1	User Login and Authentication Panel CYBER threat classifier Streamlit App	54
A.2	Enter input CTI text in streamlit dashboard	55
A.3	Prediction result of classified threat	56

List of Tables

2.1	Comparative Analysis of Recent Literature and Proposed System	15
6.1	Baseline Model Results (5-fold Cross-Validation)	40
6.2	Full Model Comparison, Sorted by Macro-F1	41
6.3	Per-Class Performance of the Proposed Hybrid Model	43

Abstract

Cyber Threat Intelligence (CTI) reporting has grown from simple indicator lists into narrative-driven text describing adversary tactics, techniques, and procedures, threat actors, and targeted infrastructure. Automatically classifying this text into actionable threat categories has become an important applied challenge, yet the datasets available for this task rarely include a ready-made category label, and published hybrid Transformer-recurrent architectures for this task consistently rely on general-purpose encoders despite domain-adapted cybersecurity encoders being shown, separately, to represent security vocabulary more effectively. This dissertation addresses both gaps directly. A genuine classification target is constructed from raw CTI text using keyword-rule matching supported by curated malware, ransomware, and threat-actor lexicons, deliberately avoiding any dependency on annotator-authored explanatory fields so the label remains learnable from exactly the information available to a deployed system. A hybrid SecureBERT-BiLSTM-Attention architecture is developed and benchmarked against classical machine learning baselines (TF-IDF with logistic regression and a support vector machine), a bidirectional LSTM trained from randomly initialised embeddings, and a published literature benchmark for an equivalent general-BERT hybrid architecture. The proposed model achieves a macro-averaged F1 score of 0.9370, exceeding the strongest classical baseline by 2.73 points and the published general-BERT hybrid benchmark by 7.7 points, directly confirming that domain adaptation compounds with the architectural benefit of a hybrid design. A diagnostic evaluation further links model performance to entity-aware structural measures computed from the dataset's existing entity annotations, identifying low-signal, structurally ambiguous records as the consistent source of remaining classification difficulty across every model evaluated.

Chapter 1

Introduction

1.1 Overview

Cyber Threat Intelligence (CTI) has become a central input to modern security operations, providing the situational detail analysts need to identify, prioritise, and respond to emerging threats. As CTI reporting has matured beyond simple indicators of compromise into narrative descriptions of adversary behaviour, automatically classifying this text into meaningful threat categories has emerged as a practical, high-value application of natural language processing within cybersecurity. This dissertation investigates whether a hybrid deep learning architecture, combining a cybersecurity domain-adapted transformer encoder with a recurrent sequence layer, can classify CTI text more effectively than existing approaches, and whether the entity annotations already present in many CTI datasets can be used analytically to explain where such a system succeeds and where it does not. The remainder of this chapter sets out the background, rationale, problem, aims, and structure of the work that follows.

1.2 Background

Natural language processing has been applied to cybersecurity text for over a decade, beginning with rule-based and information-extraction approaches to pulling structured indicators out of unstructured reports, and progressing, alongside the wider field, toward transformer-based language models. Two developments are particularly relevant to this dissertation. First, domain-adapted transformer encoders, pretrained specifically on cybersecurity-domain text rather than general internet text, have been shown to represent security-specific vocabulary, malware family names, CVE identifiers, and threat-actor terminology, more effectively than general-purpose encoders such as standard

BERT[3,4]. Second, a growing line of applied work has shown that hybrid architectures, combining a transformer encoder with a recurrent layer such as a bidirectional LSTM, outperform either component used alone on threat-text classification tasks[10]. These two developments have, until now, been investigated separately: hybrid architectures in the literature use general-purpose encoders, and domain-adapted encoders have not been tested inside a hybrid architecture. CTI datasets have also grown richer in structure, frequently including annotated entities (malware, threat-actor, vulnerability, and related types) and relations between them, alongside the underlying text, following conventions similar to the Structured Threat Information Expression (STIX) object model widely used in the threat-intelligence-sharing community.

1.3 Rationale

Three observations motivate the specific approach taken in this dissertation. First, since hybrid transformer-recurrent architectures and domain-adapted encoders have each independently been shown to improve on a general-purpose, non-hybrid baseline, combining them is a natural next step that, at the time of writing, had not been tested and reported in the published literature for CTI classification specifically. Second, entity and relation annotations, though present in many CTI datasets at no additional collection cost, are typically used only to train named-entity recognition models and are rarely turned back on the dataset itself as a descriptive or diagnostic tool, an opportunity this dissertation treats as worth pursuing rather than as an afterthought. Third, a recent systematic review of over forty papers on threat-technique extraction from CTI text found results largely incomparable across studies, due to inconsistent ontologies, non-public datasets, and differing evaluation protocols[2], motivating the transparent, directly reproducible benchmarking protocol adopted throughout this work.

1.4 Problem Statement

The dataset used in this dissertation, in common with many CTI resources, does not contain a usable document-level threat-category label: the closest candidate field is a free-text diagnostic explanation rather than a fixed category, and the annotated entity fields describe specific mentions within a record rather than that record's overall topic. A genuine classification target therefore has to be constructed before any classification model can be trained or evaluated, and constructed in a way that does not depend on information a deployed system, receiving only raw incoming text, would not have available. Separately, the dataset is imbalanced and unevenly distributed across threat categories,

properties shown during this project to measurably disadvantage models, such as a bidirectional LSTM, that must learn their representations entirely from the available training data rather than from prior pretraining. Finally, as set out in Section 1.2, no published hybrid Transformer-BiLSTM architecture for CTI or related threat-text classification has been evaluated with a domain-adapted encoder in place of a general-purpose one, leaving open whether the two independently-demonstrated sources of improvement, domain adaptation and hybrid architecture, compound, are redundant, or interact in some other way when combined.

1.5 Research Aim

The aim of this dissertation is to design, implement, and rigorously evaluate a hybrid Securebert-BiLSTM-Attention architecture that substitutes a domain-adapted cybersecurity encoder for a general-purpose one, benchmarked transparently against classical and recurrent baselines and against the published literature.

1.6 Research Objectives

To achieve the central aim of this study, the following technical objectives are pursued:

1. To construct and validate a genuine, text-derivable threat-category label for a CTI dataset that does not natively contain one.
2. To implement and benchmark classical machine learning and recurrent baseline classifiers under a consistent, stratified cross-validation protocol.
3. To perform a structured, entity-aware exploratory analysis of the dataset, computing descriptive measures (Entity Density, Entity Richness, Threat Context Score, Rare Entity Ratio) directly from existing entity annotations.
4. To design, implement, and evaluate a hybrid SecureBERT-BiLSTM-Attention architecture, and to benchmark it against the baselines above and against a comparable result reported in the published literature[10].

1.7 Research Questions

Primary research question:

Does substituting a domain-adapted cybersecurity encoder for a general-purpose transformer encoder, within an otherwise identical hybrid Securebert-BiLSTM-Attention ar-

chitecture, improve CTI threat classification performance beyond classical machine learning baselines, a from-scratch recurrent baseline, and the published literature benchmark for the equivalent general-purpose-encoder configuration[10]?Supporting questions:

- Can a genuine, text-derivable threat-category label be constructed for a CTI dataset that contains no native category field, and validated to an acceptable standard of accuracy?
- Does resolving preprocessing challenges and class imbalance through standard pipeline designs measurably affect baseline classification performance?

1.8 Detailed Technical Justification

The choice of SecureBERT as the domain-adapted encoder is justified by prior published evidence that it outperforms general-purpose RoBERTa and BERT checkpoints specifically on cybersecurity-domain language modelling and downstream tasks[3], making it a reasonable representative of the domain-adaptation approach rather than an arbitrary selection. The bidirectional LSTM layer positioned after the encoder is justified on the basis that it re-models sequential dependency over the encoder's already-contextualised token embeddings, a combination with precedent in adjacent domains such as biomedical and legal named-entity recognition[9], rather than being a novel or untested architectural pattern. The additive attention layer is justified both by its function, producing a learned, weighted summary of the sequence rather than relying on a fixed pooling operation, and by the interpretability it affords as a secondary benefit, since attention weights indicate which tokens contributed most to a given classification decision. Freezing the lower encoder layers during fine-tuning is justified directly by an empirical finding from this project's own baseline results, namely that a bidirectional LSTM trained entirely from randomly initialised weights underperforms substantially on a dataset of this scale, indicating that components required to learn extensively from scratch are disadvantaged at this sample size; freezing reduces the number of parameters the hybrid model must similarly learn from scratch. Stratified five-fold cross-validation, applied uniformly across every model in the comparison, is justified by the combination of a moderate dataset size and class imbalance, under which a single train-test split would be highly sensitive to which records happened to fall into each partition.

1.9 Extended Technical Discussion

Beyond the core architectural choices, several further methodological decisions merit discussion. The threat-category label is constructed using text alone, not the dataset's

diagnostic explanation field, a decision made after directly measuring that a substantial proportion of labels would otherwise depend on information absent from the raw text a deployed system would receive, a form of validity gap rather than a purely technical concern. The evaluation protocol benchmarks the proposed model against a figure reported in the published literature for a comparable general-purpose-encoder hybrid[10], rather than against a second, independently retrained general-encoder hybrid on this project's own dataset; this is a deliberate scope decision, made to keep the computational cost of the evaluation proportionate to the project's timeframe, and is treated as a limitation to be stated explicitly rather than concealed.

1.10 Research Significance

This dissertation contributes to the literature by directly testing a combination, domain-adapted encoder within a hybrid Transformer-recurrent architecture, that prior published work had investigated only separately, and by reporting the result transparently against both an internal baseline suite and an external literature benchmark[10]. It contributes methodologically by demonstrating a validated approach to constructing a classification label where a dataset provides none natively, including the discovery and correction of a label-construction dependency that would otherwise have compromised the resulting model's real-world validity. It contributes an evaluation approach that goes beyond a single aggregate performance figure, linking entity-aware structural measures to per-model error patterns in a way intended to be reusable on other CTI datasets that share the same entity-annotation structure. Practically, a classifier of this kind supports the triage stage of security operations, where the volume of incoming threat reporting routinely exceeds what analysts can review manually, and a reliable, well-characterised automated first pass, including an explicit account of which kinds of report it is least reliable on, has direct operational value.

1.11 Research Structure

The remainder of this dissertation is organised as follows. Chapter 2 reviews the literature on natural language processing for cybersecurity, domain-adapted language models, hybrid transformer-recurrent architectures, and CTI extraction and classification more broadly, identifying the specific gap this dissertation addresses. Chapter 3 sets out the methodology, covering dataset preparation, label construction and validation, feature engineering, the experimental design and evaluation protocol, and the baseline and proposed model architectures. Chapter 4 describes the system design and architecture of the

implemented pipeline in detail. Chapter 5 documents the implementation itself, including the technology stack, key implementation decisions, and challenges encountered and resolved during development. Chapter 6 presents the results, benchmarking the proposed hybrid model against the baselines and the literature, together with per-class and diagnostic analyses. Chapter 7 discusses these results in relation to the research questions set out in Section 1.7, alongside the limitations of the work, and outlines future work and conclusions.

Chapter 2

Literature Review

2.1 Introduction

Cyber Threat Intelligence (CTI) has developed from a collection of simple Indicators of Compromise (IOCs) to a narrative-driven reporting of adversary Tactics, Techniques and Procedures (TTPs), threat actors, targeted infrastructure and recommended defensive actions[1],2]. The volume and complexity of today's CTI text has outstripped the ability of manual analyst review, and Security Operations Centres (SOCs) and Computer Security Incident Response Teams (CSIRTs) rely on timely triage of this reporting to prioritise defensive action. The classification of CTI narratives into actionable threat categories has thus emerged as an urgent applied challenge in both academic and operational security research. Modern CTI datasets often include more than just a narrative text: each record is enriched with structured entities (e.g., attack-pattern, malware, threat-actor, campaign, identity, infrastructure, vulnerability, tools, URL, and location) and explicit relation tuples between these entities. This annotation style is similar to the Structured Threat Information Expression (STIX) object model that is widely adopted by the threat-intelligence-sharing community, and to recent academic resources like AnnoCTR, which connects entities, tactics, and techniques in real CTI reports. These annotations are a valuable and underutilized source of signal. They are typically used for named entity recognition alone, but are not often used to describe the composition of a dataset or to explain the success or failure of a classifier on a particular record. This dissertation does not view that gap as an afterthought, but rather an opportunity, and entity and relation structure as an analytical lens, not as raw input to a classifier. Meanwhile, there are tensions on the modelling side of CTI automation as well. Security text is under-performed by general-purpose transformer encoders due to the presence of specialised vocabulary (CVE identifiers, malware family names, and TTP-specific phrasing), which motivated a family of domain-adapted encoders like SecureBERT[2] and CySecBERT[3]. As the adop-

tion of Large Language Models (LLMs) for CTI tasks has accelerated, there has been increasing evidence of unreliable attribution and hallucination when using LLM models without grounding[4]. In between these two extremes lies a design space of hybrid, well-balanced architectures that leverage the representation power of transformer models with sequence modelling and attention, but are still easily explainable and auditable. This dissertation fills that gap.

2.2 Foundational NLP Architectures

Modern text classification in cybersecurity, as elsewhere in NLP, rests on the transformer architecture introduced by Vaswani et al.[5]. Unlike recurrent models, the transformer processes an entire sequence in parallel and models dependencies between tokens through self-attention, in which each token representation is updated as a weighted combination of all other tokens in the sequence.

where Q , K , and V are learned linear projections (query, key, value) of the input embeddings and d_k is the key dimension, with the scaling factor d_k preventing the dot products from growing too large and saturating the softmax. Multiple attention "heads" are computed in parallel and concatenated, allowing the model to attend to different types of relationship (e.g., syntactic proximity versus long-range semantic association) simultaneously. Because self-attention has no inherent notion of token order, positional encodings are added to the input embeddings to inject sequence position information. BERT[6]built on this architecture by pretraining a bidirectional transformer encoder using masked language modelling (predicting randomly masked tokens from both left and right context) together with a next-sentence-prediction objective, producing contextual embeddings that transfer well to downstream classification tasks via fine-tuning. BERT and its derivatives now underlie the majority of modern text-classification systems, including those applied to cybersecurity text, largely because bidirectional context is well suited to disambiguating domain terms that depend on surrounding text (for example, distinguishing a CVE identifier used descriptively from one used as an indicator of active exploitation).Recurrent architectures remain relevant despite the dominance of transformers, particularly in hybrid pipelines. The Long Short-Term Memory network (LSTM;[7]) addresses the vanishing-gradient problem of simple recurrent networks through a gated cell state, with input, forget, and output gates controlling what information is written to, retained in, and read from the cell at each timestep.[13] Supports the use of LSTM with historical load and weather/time-related variables.A Bidirectional LSTM (BiLSTM) runs two such LSTMs over a sequence (one left-to-right and one right-to-left) and concatenates their hidden states, giving each position access to both preceding and following context in a manner that complements, rather than duplicates, what a transformer en-

coder already [8] BiLSTM layer exists in adjacent domains: Jiang et al.[9] propose a BERT-BiLSTM-CRF architecture for Chinese electronic medical record named-entity recognition, in which BERT contextual embeddings are passed through a BiLSTM to model sequential label dependencies before a Conditional Random Field layer performs structured prediction. While the present dissertation's task is sequence-level classification rather than sequence labelling (and therefore does not require a CRF layer), the underlying rationale (that a BiLSTM layer can usefully remodel sequential dependency over already-contextualised transformer embeddings) directly motivates the hybrid architecture proposed in Chapter 4. It is worth noting a practical limitation of the self-attention mechanism that is directly relevant to CTI text: computing pairwise attention scores across all token pairs incurs $O(n^2)$ time and memory complexity in sequence length n , which constrains the maximum practical input length for transformer encoders applied to long-form CTI narratives without truncation or chunking. CTI reports vary considerably in length (from short indicator summaries to multi-paragraph incident writeups) so this quadratic cost is a genuine, if manageable, engineering constraint on the encoder design discussed in Chapter 4, rather than a purely theoretical concern. Earlier, non-transformer embedding approaches such as static word vectors and the contextual, character-aware embeddings introduced by ELMo predate BERT and do not suffer this quadratic cost, but they lack the deep bidirectional contextualization that later made transformer encoders the default choice for classification tasks; they are noted here for historical completeness rather than as candidates for the present architecture.

2.3 Domain-Adapted Language Models for Cybersecurity

General-purpose language models underperform on cybersecurity text for a well-documented reason: their vocabularies and pretraining corpora contain little of the specialised terminology that dominates security narratives, including CVE and CWE identifiers, malware family names, protocol and tool names, and TTP-specific phrasing drawn from frameworks such as MITRE ATT&CK. Subword tokenisers trained on general corpora tend to fragment such terms into uninformative pieces, degrading the quality of the resulting contextual embeddings. This limitation has motivated a line of domain-adapted encoders trained or further pretrained specifically on cybersecurity corpora. SecureBERT [2] adapts the RoBERTa architecture using a custom tokeniser and continued pretraining on a large corpus of cybersecurity text, improving performance on downstream tasks such as named entity recognition and text classification relative to general BERT/RoBERTa checkpoints. CySecBERT [3] pursues a similar domain-adaptation strategy indepen-

dently, further pretraining BERT on a large, curated cybersecurity corpus and reporting consistent gains over general-domain BERT across several cybersecurity NLP benchmarks. Across all efforts, the consistent finding is that domain-adapted encoders outperform general-purpose transformers on downstream CTI tasks, which directly motivates this dissertation's choice to use a domain-adapted transformer (rather than a general-domain one) as the encoder backbone for both a standalone baseline and the proposed hybrid model. A common thread across these domain-adaptation efforts is the treatment of tokenisation itself as a design decision rather than a fixed preprocessing step. General-purpose subword tokenisers, trained on web and encyclopaedic text, tend to split security-specific tokens (a CVE identifier, a malware family name, a living-off-the-land binary name) into fragments that carry little standalone meaning, forcing the encoder to reconstruct domain semantics from noisy subword pieces. Domain-adapted tokenisers, by contrast, are trained or extended on cybersecurity corpora so that such tokens are more often preserved as coherent units, which in turn improves the quality of the contextual embeddings the encoder produces. An early pioneer in this space was CyBERT [16], which demonstrated the viability of fine-tuning base BERT encoders on open-source unstructured and semi-unstructured threat reports to extract domain-specific entities in Security Operations Center (SOC) workflows. More recently, this domain-adaptation lineage has culminated in SecureBERT 2.0 [10], which transitions from the standard RoBERTa/BERT base to the ModernBERT architecture. SecureBERT 2.0 introduces advanced long-context modeling and hierarchical encoding capabilities to process highly complex and heterogeneous documents (such as source code and raw threat reports) while being pretrained on a cybersecurity corpus exceeding 13 billion text tokens and 53 million code tokens, establishing a new state-of-the-art for semantic search, entity extraction, and vulnerability detection.

2.4 Hybrid Transformer–Recurrent Architectures for Threat Text Classification

A more recent and directly relevant line of work combines transformer encoders with recurrent or graph-based layers specifically for threat-text classification and extraction, rather than using either family of model in isolation. Hidayatulloh et al. [11] report that a hybrid BERT-BiLSTM architecture, in which BERT-derived token embeddings are passed through a BiLSTM layer before classification, improves macro F1 from 0.82 for standalone fine-tuned BERT to 0.86 on a CTI text classification task, outperforming logistic regression (0.69), SVM (0.72), and standalone LSTM (0.75) baselines under the same evaluation protocol. Xiang, Shi, and Zhang [12] combine BERT with a BiGRU-CRF

layer for Advanced Persistent Threat (APT) event extraction from web text, reporting the highest F1 score among all compared architectures, including standalone BERT and ERNIE. Liu and Dai [13] apply a hybrid BERT-LSTM network to SQL injection attack detection, again outperforming standalone baselines by dynamically extracting contextual features that neither model captures alone. Ferrag et al. [14] show that a lightweight, purpose-built BERT variant (SecurityBERT) outperforms CNN- and RNN-based methods, including prior hybrid GAN-Transformer and CNN-LSTM solutions, on IoT/IIoT threat detection, achieving 98.2 percent accuracy across fourteen attack types with sub-150-millisecond inference time. Across all five of these studies, the encoder component is either a general-purpose BERT checkpoint fine-tuned directly on the task, or a purpose-built model trained from scratch; none evaluate an existing, separately validated domain-adapted cybersecurity encoder, such as SecureBERT [2], as a drop-in replacement within the hybrid architecture. This is a notable gap given that domain-adapted encoders have been shown, independently, to represent cybersecurity vocabulary more effectively than general-purpose transformers. Whether this domain-adaptation advantage compounds with the architectural advantage of a hybrid transformer-recurrent design, rather than being redundant with it, has not previously been tested. This dissertation addresses that question directly, by replicating the general-BERT hybrid configuration reported by Hidayatulloh et al. [11] as a literature baseline, then substituting SecureBERT for BERT within an otherwise identical architecture. This design choice is also supported by recent empirical evaluations. For example, Kaur et al. [15] conducted a comprehensive comparative analysis of standalone LSTM and Transformer architectures for threat classification, demonstrating that while LSTM sequence modeling captures short-term local semantics, transformer attention is vastly superior in modeling long-range contextual dependencies within multi-stage threat narratives. Fine-tuning a hybrid transformer-recurrent design—wherein the sequential recurrent layer refines the deep, multi-head attention contextual embeddings of a domain-specific encoder—thus combines the unique strengths of both paradigms while mitigating their individual limitations.

2.5 TTP and Entity/Relation Extraction from CTI Text

A substantial and longrunning body of work addresses the extraction of structured information (entities, relations, and TTPs) from unstructured CTI reports. TTPDrill [1] was an early and influential system in this space, combining information extraction techniques with an ontology mapping threat actions expressed in free text to a structured threat action representation, enabling automatic, ontologygrounded extraction of adver-

sarial behaviour from prose reports. CASIE [16] extended this direction to cybersecurity event extraction, identifying event triggers and arguments (e.g., attack, vulnerability-related, or ransom events) within CTI text using a neural sequence labelling approach. The AnnoCTR dataset provides a large, richly annotated resource linking entities, tactics, and techniques within genuine CTI reports, offering a valuable benchmark for entity and TTP linking tasks and a structural precedent for the dataset schema used in this dissertation. Despite this sustained research activity, a recent systematic review by Büchel et al. [17] (a "Systematisation of Knowledge" (SoK) covering over forty papers on automated TTP extraction) concludes that reported results are largely incomparable across studies. The review attributes this to inconsistent ontologies (different papers ground extraction in different, incompatible taxonomies), nonpublic or inconsistently partitioned datasets, and differing evaluation protocols, and it explicitly questions whether the field can currently demonstrate genuine cumulative progress. This is a methodological gap that the present dissertation addresses directly, by adopting transparent, reproducible baselines evaluated under a single consistent protocol on one dataset, rather than by contributing yet another incomparable point estimate to the literature. Comparing these systems along a common set of dimensions is itself instructive. TTPDrill and CASIE both predate the widespread availability of large pretrained transformers and rely respectively on ontology-driven information extraction and neural sequence labelling over comparatively narrow, task-specific corpora. AnnoCTR is more recent and incorporates transformer-based representations, reflecting the same broader shift toward pretrained encoders documented in Section 2.3, and is explicitly constructed as a benchmark dataset linking entities, tactics, and techniques for others to train and evaluate against. None of the three, however, use the entity or relation annotations they produce to describe dataset composition beyond basic frequency counts, and none link structural annotation properties to downstream error analysis (the specific combination this dissertation pursues). A further, more specific observation from this strand of literature motivates the design of the present study: although entity and relation annotations are common in CTI datasets (precisely because they are needed to support TTP and event extraction research) they are rarely used to characterise dataset composition or to explain model behaviour once extraction or classification has been performed. Existing work treats entities and relations almost exclusively as extraction targets or as intermediate representations feeding a downstream extraction pipeline, not as descriptive or diagnostic signal in their own right. The difficulty of mapping raw narratives to structured technical indicators under low-resource and imbalanced conditions remains a major challenge. To address this, Park and You [18] proposed a two-stage training methodology to classify CTI sentences into MITRE ATT&CK techniques, demonstrating that pretraining on auxiliary augmented datasets before fine-tuning on primary reports significantly boosts Macro-F1 scores on low-resource attack patterns. From an operational gathering perspective, automated

threat triage also depends on efficiently locating relevant CTI documents. In this area, ThreatCrawl [19] introduced a BERT-based focused crawling framework that dynamically adapts its crawling path to retrieve highly relevant Open Source Intelligence (OSINT) and threat articles from the web, illustrating how document-level BERT classifiers can be integrated into active incident ingestion pipelines.

2.6 Large Language Models in CTI and Reliability Concerns

The recent proliferation of general-purpose LLMs has led to their direct application to CTI tasks, including summarisation, classification, and TTP identification via prompting. This uptake has, however, exposed a mismatch between general LLM evaluation and the demands of CTI reasoning: CTIBench[4] was explicitly constructed because standard LLM benchmarks fail to capture CTI-specific reasoning tasks such as threat actor attribution, vulnerability prioritisation, and TTP mapping, and its results highlight that hallucination and unreliable attribution remain persistent risks when generative models are used without external grounding (a finding echoed across the broader LLM-for-security literature). In practical terms, this means that an LLM asked to classify or summarise a CTI report may generate a plausible-sounding but factually unsupported threat attribution, a failure mode that is particularly costly in a security context where downstream defensive action may depend on the output. These reliability concerns motivate the present dissertation's choice of architecture. Rather than relying on an ungrounded generative LLM pipeline for classification, this work adopts a smaller, supervised, well benchmarked hybrid model whose behaviour can be evaluated exhaustively against held-out labelled data and whose errors can be diagnostically traced back to measurable data characteristics (a degree of auditability that prompted, ungrounded LLM classification does not currently offer).

2.7 Identified Research Gaps

Synthesising the literature patterns reveals several concrete gaps that this dissertation is positioned to address:

1. No domain-adapted encoder tested in a hybrid architecture. Published hybrid transformer-recurrent CTI and threat-text classifiers[10,11,12] all use general-purpose BERT as the encoder; none test a domain-adapted cybersecurity encoder such as SecureBERT within the same architecture, despite domain adaptation being separately shown to outperform

general-purpose transformers on cybersecurity text.

2. Shallow use of entity/relation annotations. CTI datasets that include entity and relation annotations alongside text rarely use them for anything beyond named entity recognition itself; they are seldom used to characterise dataset composition or difficulty.
3. Fragmented, incomparable evaluation. A systematic review of over forty TTP extraction papers found results largely incomparable across studies due to custom ontologies and inconsistent protocols[17]], leaving open whether the field is genuinely progressing.
4. Unverified LLM reliability. LLM-based CTI systems show persistent hallucination and attribution errors, motivating benchmarks built specifically to expose this[4], yet many applied systems still deploy LLMs without grounding or a simpler, well-benchmarked alternative.
5. Accuracy-only evaluation. Reported results commonly emphasise overall accuracy while omitting per-class performance, obscuring weaknesses on less frequent threat categories. The identified gaps map directly to the experimental design devel-

Table 2.1: Comparative Analysis of Recent Literature and Proposed System

Study/System	Encoder Used	Sequence Layer	Dataset Size	Entity-Aware Analysis
Hidayatulloh et al. [11]	General BERT	BiLSTM	CTI Narratives	No
Xiang et al. [12]	General BERT	BiGRU-CRF	Web Threat Data	No
Satyapanich et al. [16]	None (Static Vectors)	BiLSTM-CRF	CASIE Corpus	No
Proposed System	SecureBERT (Domain-Adapted)	BiLSTM-Attention	1,420	Yes (Full Evaluation)

oped in Chapter 3. In particular, the study tests the domain-adaptation gap through a controlled general-BERT versus SecureBERT hybrid comparison, addresses shallow use of annotations through entity-aware EDA and diagnostic analysis, improves comparability through a common stratified cross-validation protocol, avoids relying on ungrounded generative classification, and reports macro and per-class results rather than accuracy alone.

Chapter 3

Methodology

3.1 Overview

The study follows a quantitative, experimental and comparative research design. Each CTI record is assigned one document-level threat category. The workflow progresses from preprocessing and target construction to exploratory data analysis (EDA), model training, and evaluation. TF-IDF with Logistic Regression and SVM provide lexical baselines; BiLSTM provides a recurrent baseline; and Transformer–BiLSTM–Attention models provide the hybrid comparison. This progression isolates the contribution of lexical representation, sequence modelling, contextual pretraining and cybersecurity-domain adaptation.



Figure 3.1: The methodology workflow for the CTI classification system

3.2 Dataset and Data Structure

The dataset contains 1,420 records and 17 original columns. Each record includes an identifier, CTI text, relations, diagnosis, solutions, and up to three annotated entity slots containing identifiers, labels and character-level offsets. Available annotations include malware, threat actors, attack patterns, infrastructure, software, vulnerabilities, tools, campaigns, identities, URLs, locations, hashes, file paths, time expressions, IPv4 addresses, registry keys and email addresses. The dataset therefore combines narrative text with structured cybersecurity annotations.

The entity and relation annotations are not fused directly into the principal classifier input. Instead, they are used for exploratory and diagnostic analysis. This methodological separation is intentional. Directly feeding entity-derived features into a classifier whose target is itself constructed from textual threat indicators could make it difficult to distinguish genuine contextual learning from label-related shortcuts. The annotations therefore function as an analytical lens for examining the data and explaining model behaviour.

3.3 Data Cleaning and Preprocessing

The preprocessing stage checks dimensions, data types, missing values, duplicate identifiers and structural consistency. Optional entity slots may be missing; missing identifiers and offsets use sentinel values and missing labels use NONE. The single missing diagnosis value is handled safely when text and diagnosis are combined. Text cleaning is conservative so that CVE identifiers, URLs, hashes and malware names are retained. Entity labels are stripped and normalised to uppercase, and the relations field is parsed for structural validation and descriptive analysis. Entity labels are stripped of surrounding whitespace and converted to a consistent uppercase representation. This avoids treating formatting variations such as URL and url as different categories. Numeric identifier and offset columns are converted to appropriate numeric types. The relations field is parsed sufficiently to validate its structure and support relation-level descriptive analysis. These operations are applied before model construction so that every experimental stage receives a consistent and reproducible representation.

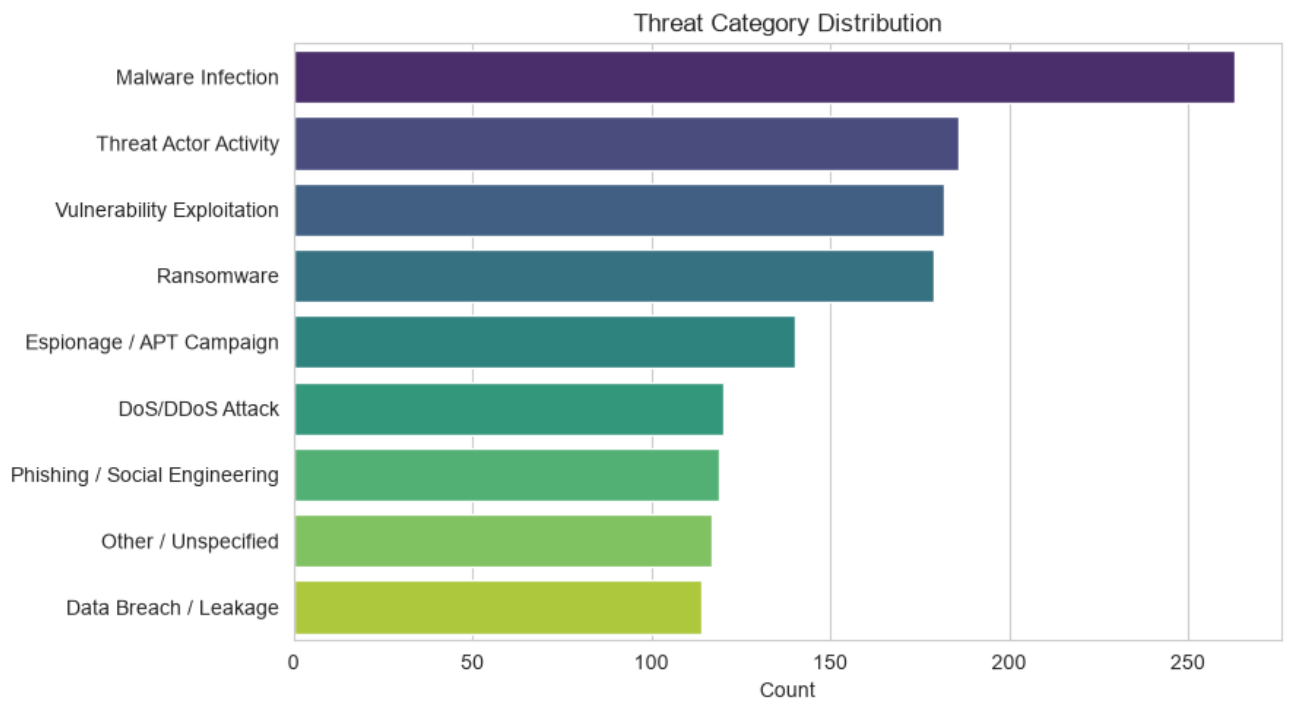


Figure 3.2: Class distribution

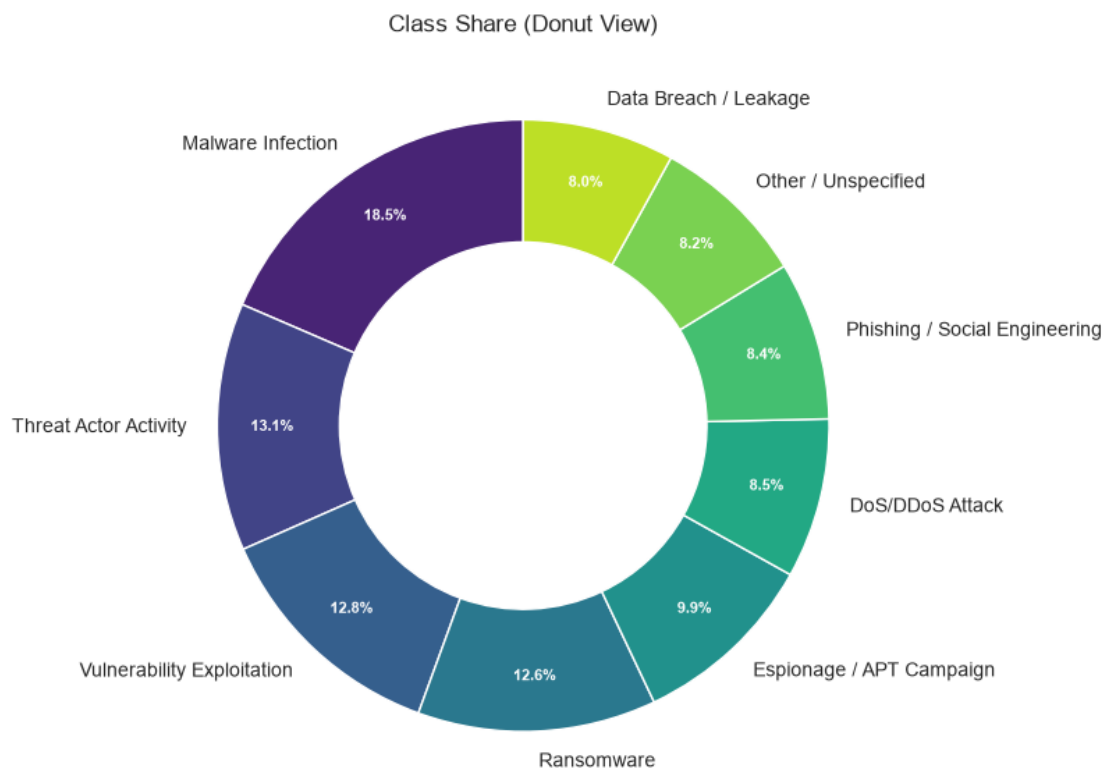


Figure 3.3: category distribution donut

3.4 Construction of the Classification Target

The dataset does not provide a native document-level threat category, so a supervised target is constructed from the CTI text and diagnosis fields. The combined representation is normalised and processed using priority-based keyword rules. Nine categories are produced: DoS/DDoS Attack, Ransomware, Phishing / Social Engineering, Vulnerability Exploitation, Espionage / APT Campaign, Data Breach / Leakage, Malware Infection, Threat Actor Activity, and Other / Unspecified. Priority ordering handles overlapping indicators by checking specific categories before broader ones. A reproducible sample of 50 records is exported for manual inspection because the target is derived rather than human-labelled. The rules are not reported as an independent predictive baseline because evaluating them against labels generated by the same rules would be circular. Priority ordering is necessary because one record can contain several overlapping security indicators. Specific categories are checked before broader categories so that, for example, ransomware terminology is not automatically absorbed into the general malware category. Vulnerability and CVE indicators are similarly considered before broad threat-actor terminology. Records that do not satisfy any category-specific rule are assigned to Other / Unspecified. Because the target is derived rather than human-labelled, a reproducible sample of 50 records is exported for manual inspection. This quality-control stage is used to assess whether the generated labels reasonably represent the underlying CTI narrative. The keyword rules are therefore treated as a target-construction mechanism rather than as an independent predictive baseline, because evaluating those same rules against labels generated by them would be circular.

3.5 Entity-Aware Exploratory Features

EDA derives record-level measures from the annotations. Entity Count measures available entities; Entity Density normalises entity count by text length; and Entity Richness counts unique entity types. Threat Context Score weights entity concentration, with additional emphasis on THREAT-ACTOR, CAMPAIGN, VULNERABILITY and ATTACK-PATTERN. Rare Entity Ratio measures the proportion of entities from relatively infrequent types. These measures support EDA and error analysis rather than direct classifier input. A Threat Context Score is calculated as a composite measure of entity concentration. Each entity contributes a base score, while high-signal categories such as THREAT-ACTOR, CAMPAIGN, VULNERABILITY and ATTACK-PATTERN receive additional weighting. The score is used as a contextual-information proxy rather than being interpreted as an official security severity rating. Rare Entity Ratio measures the proportion of entities associated with relatively infrequent entity types, with the rar-

ity threshold derived from the observed distribution rather than manually selected. These variables are stored for EDA and later error analysis rather than used as direct predictors in the main text-classification models.

3.6 Text-Level Statistical Features

Four text statistics are generated: word count, character count, average word length and lexical diversity. These measures describe record length and vocabulary variation and support comparison of text complexity across threat categories and prediction outcomes.

3.7 Exploratory Data Analysis

EDA examines class distribution, text length and entity characteristics before model comparison. Class plots identify imbalance; histograms and boxplots show text-length variation; IQR inspection flags unusual records without automatic deletion; and word-cloud and entity-frequency plots summarise terminology. Further analysis compares Entity Density, Threat Context Score and Rare Entity Ratio across categories using correlation heatmaps and scatter plots. Entity and relation patterns are inspected descriptively rather than used in a graph neural network.

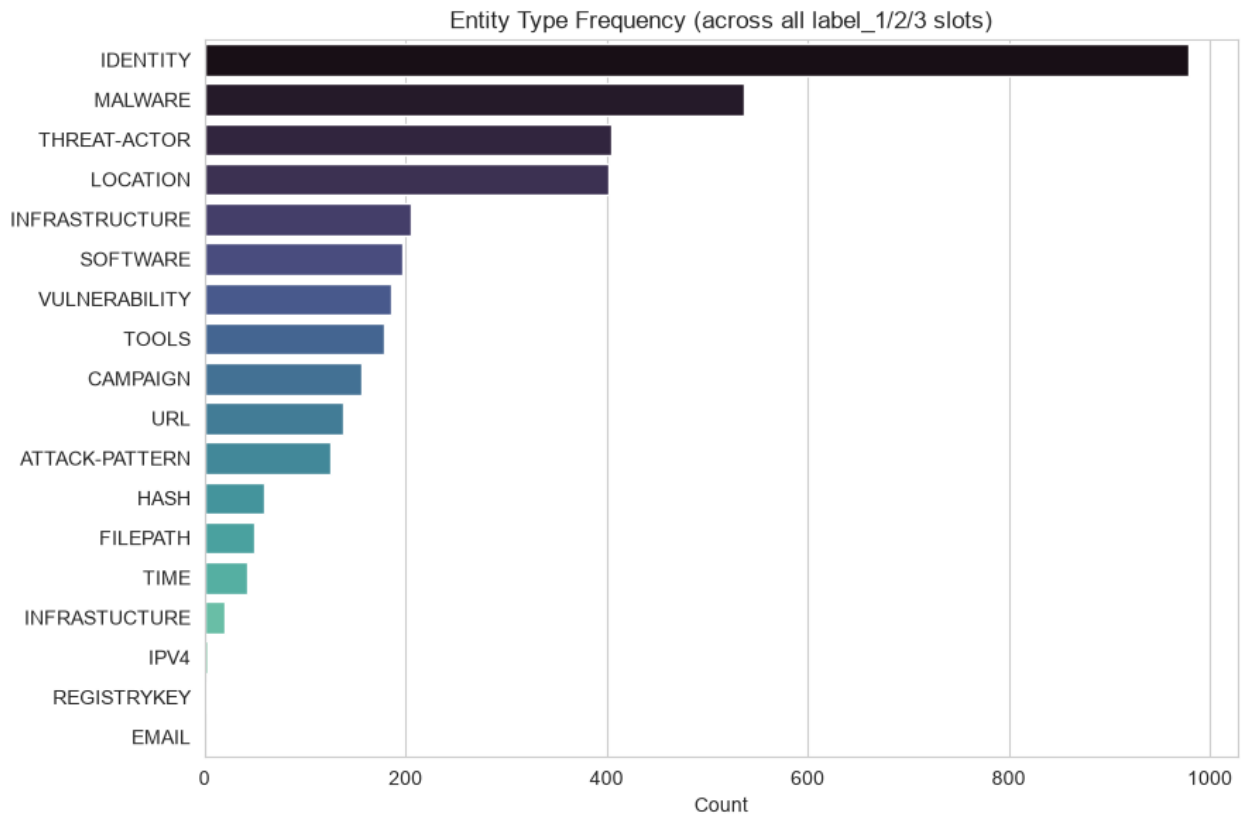


Figure 3.6: entity_type_frequency

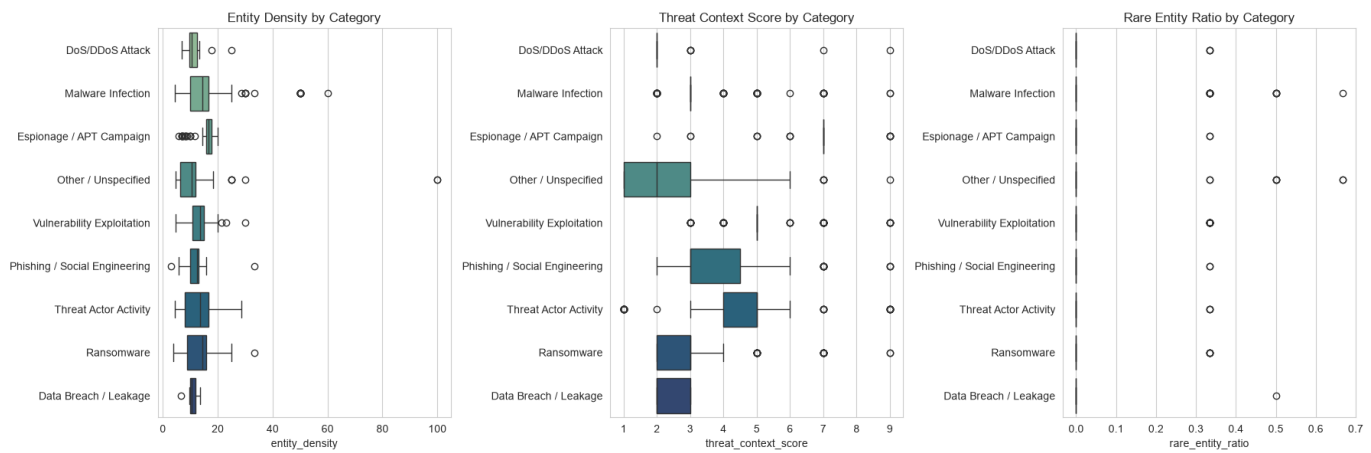


Figure 3.7: eda_features_by_category

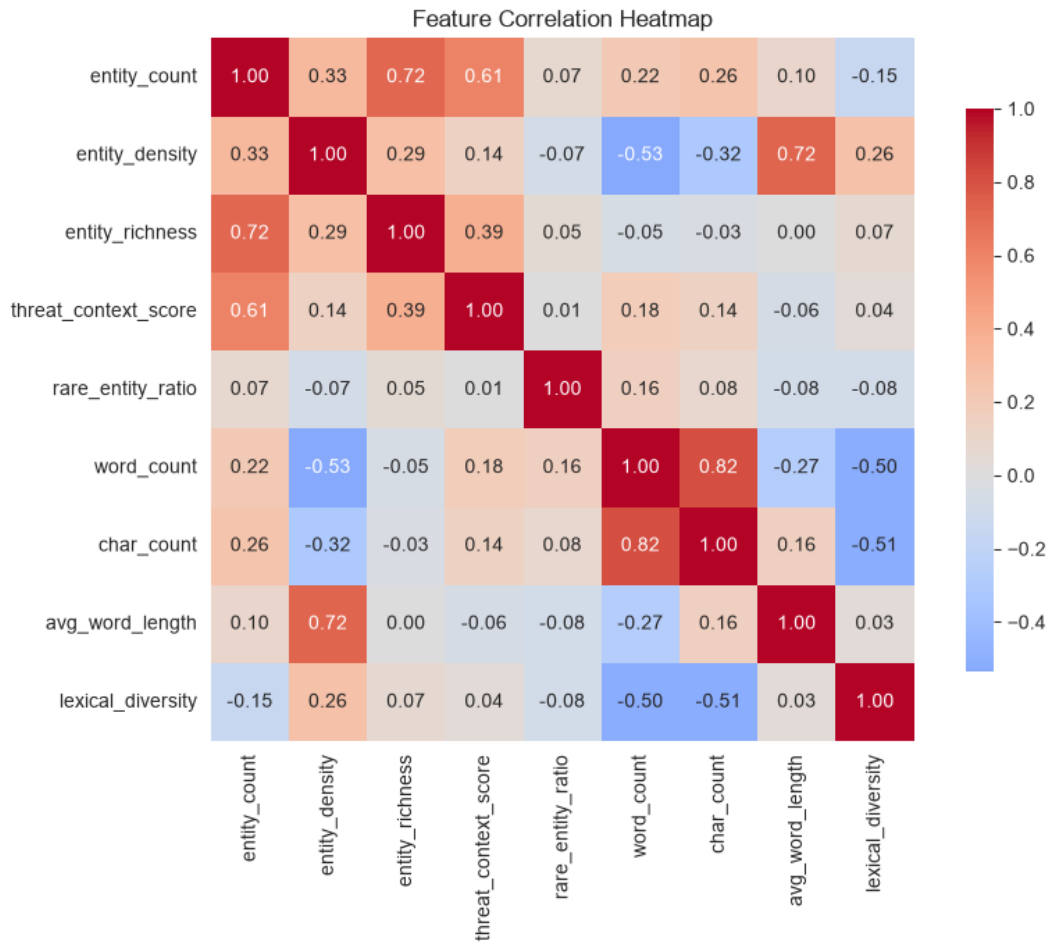


Figure 3.8: feature_correlation

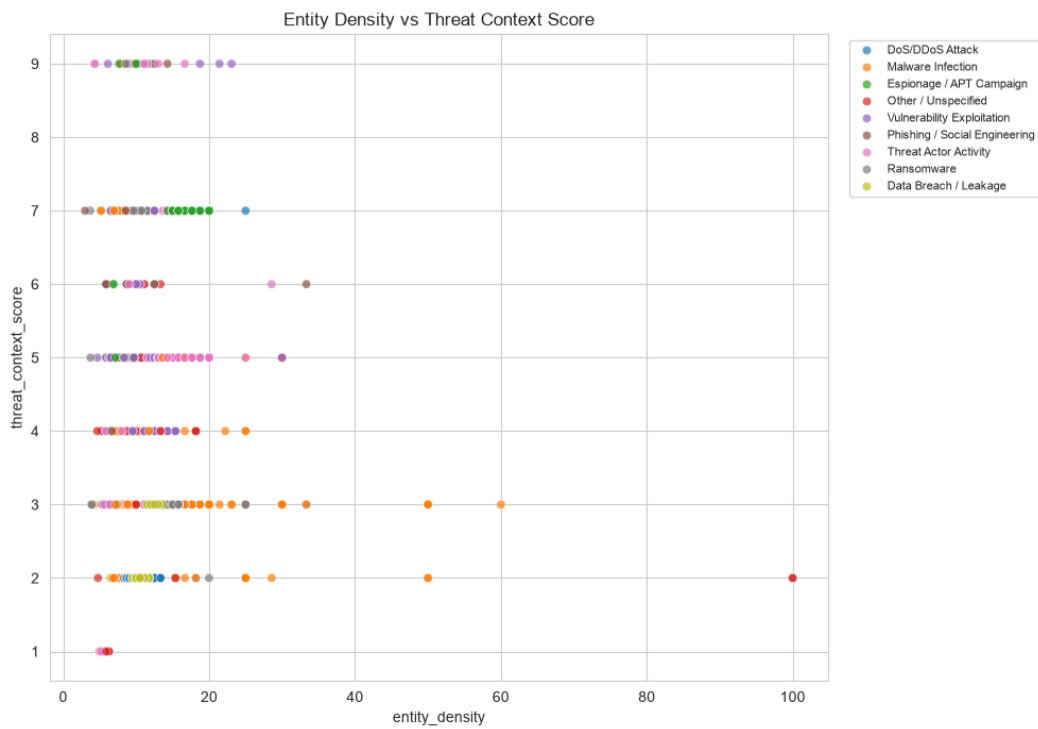


Figure 3.9: density_vs_context_scatter

Further analysis compares Entity Density, Threat Context Score and Rare Entity Ratio across categories. A correlation heatmap examines relationships among the numerical diagnostic variables, while scatter plots investigate whether entity concentration and contextual complexity vary by threat class. Entity interaction patterns are also examined descriptively using the supplied annotations and relations. The purpose is not to construct a graph neural network, but to determine whether particular combinations of security entities characterise more complex records. The EDA outputs subsequently provide the variables used to stratify diagnostic error analysis.

3.8 Model Training and Validation

Stratified five-fold cross-validation is used because the constructed threat categories are imbalanced. Stratification preserves approximately similar class proportions in each fold and reduces the risk that minority categories are poorly represented. This is especially important in cybersecurity classification, where rare threat categories can be operationally significant even when their sample size is limited. A fixed seed of 42 supports reproducibility. Out-of-fold predictions are retained so that each record is predicted by a model that did not train on that record. This design improves the credibility of evaluation because it avoids training-test leakage at the record level. The same fold structure is used across competing models to ensure a fair comparison. Performance is evaluated with metrics that reflect class imbalance and multi-class classification behavior. Macro-F1 is treated as the principal metric because it calculates F1 independently for each class and then gives equal weight to every class. This prevents majority categories from dominating the evaluation. Accuracy is less informative on its own, because a model can appear strong overall while performing poorly on rare but important threat categories.

3.9 Classical Machine Learning Baselines

The classical baseline stage uses TF-IDF unigrams and bigrams with a maximum vocabulary of 3,000 features. TF-IDF is appropriate because CTI documents often contain highly informative keywords, entity names, and attack indicators that can be captured effectively through sparse lexical representation. Logistic Regression is used as an interpretable lexical baseline with balanced class weighting and up to 1,000 iterations. This model tests whether linear decision boundaries over sparse text features are sufficient for the classification task. A linear Support Vector Machine with balanced class weighting provides a second benchmark and evaluates whether stronger margin-based separation improves performance. These models are valuable because they establish a strong and

efficient reference point. If a deep learning model fails to outperform them, then the additional complexity may not be justified. Conversely, if contextual models significantly improve macro-F1, the results would support the argument that CTI classification benefits from deeper semantic representations.

3.10 BiLSTM Deep Learning Baseline

A Bidirectional Long Short-Term Memory network is implemented as a deep learning baseline without pretrained weights. The vocabulary is built from the dataset, records are mapped to integer sequences, and the input length is limited and padded to 60 tokens. The network uses a 64-dimensional embedding layer, a bidirectional LSTM with hidden dimension 64, mean pooling, dropout, and a fully connected classification layer. Training uses Adam with learning rate 0.001, weight decay 1e-5, batch size 16, and 15 epochs. This setup tests whether sequence modeling can capture word-order dependencies and contextual cues beyond what TF-IDF can represent. BiLSTM is especially relevant in CTI texts because threat descriptions often depend on local context, such as whether a term refers to an attacker, a tool, or a victim. The BiLSTM baseline is also useful as a bridge between classical machine learning and transformer-based modeling. It evaluates whether a learnable sequence representation provides a meaningful advantage even without large-scale external pretraining.

3.11 Transformer and Hybrid Architecture

The advanced stage is configured around BERT and SecureBERT. Transformer inputs use a maximum sequence length of 64 with attention masks. The classifier applies mean pooling over valid token representations followed by dropout and a linear head. The configured training uses AdamW, learning rate 2e-5, five epochs, and batch size 8. The first eight encoder layers are frozen to reduce computational cost and overfitting risk. The proposed hybrid architecture extends the transformer encoder with a bidirectional LSTM, attention mechanism, and classification head. Contextual transformer outputs are processed sequentially by the BiLSTM, while attention weights emphasize informative tokens before classification. The hybrid uses an LSTM hidden dimension of 128. The notebook also provides a general-BERT hybrid for comparison and identifies the SecureBERT hybrid as the proposed model.

This transformer-based section is important because it introduces cybersecurity-domain language modelling into the pipeline. SecureBERT is conceptually attractive for CTI because it is designed to better capture domain-specific terminology than a general-

purpose encoder[2]. The hybrid architecture further strengthens this idea by combining pretrained contextual embeddings with sequence-level refinement and attention-based focus.

The transformer sections are configured for GPU-enabled Colab and require Hugging Face dependencies, so the methodology does not assume numerical results until those cells are executed. This keeps the chapter method-focused rather than result-dependent.

3.12 Evaluation and Error Analysis

Macro-F1 is the principal metric because the task contains multiple categories and class imbalance. It calculates F1 independently for each class and then gives equal weight to every class. For each model, mean macro-F1 and standard deviation across five folds are reported. Out-of-fold predictions support classification reports and confusion-matrix analysis for the best-performing model. Diagnostic error analysis examines prediction correctness across `rare_entity_ratio` and `threat_context_score` bands. This tests whether structurally unusual or information-dense CTI records are harder to classify and links EDA findings to model behavior. Models are ranked by mean macro-F1, and the highest-performing model is selected as the experimental candidate for subsequent project stages.

3.13 Reproducibility and Experimental Workflow

The workflow is implemented as a sequential Google Colab/Jupyter Notebook using Pandas, NumPy, Matplotlib, Seaborn, Scikit-learn, PyTorch, and Hugging Face Transformers for the transformer stage. The pipeline covers loading, validation, missing-value handling, text normalisation, entity-label standardisation, relation parsing, rule-based target construction, feature engineering, EDA, stratified five-fold validation, baseline training, deep learning, transformer experimentation, model comparison, and error analysis. Feature-engineered data and EDA outputs are saved to support reproducibility. This is important because dissertation work should be inspectable and repeatable rather than dependent on hidden manual steps. Reproducibility also supports academic transparency by making the transformation from raw CTI records to evaluation-ready features explicit. The methodology clearly separates dataset-provided information from derived information. Entity annotations are treated as structured domain knowledge, while the document-level threat category is explicitly identified as a constructed target. This provides a transparent experimental basis for comparing lexical, sequential, and cybersecurity-domain contextual representations while connecting EDA-driven feature engineering to measurable model behavior.

3.14 Methodological Considerations and Scope

Several methodological decisions are intentionally constrained to keep the experiment reproducible and proportionate to the available dataset. First, the relations field is parsed only for structural validation and relation-count inspection; graph neural networks and relation-derived predictors are outside the present scope. Second, the engineered `threat_context_score` is treated as a research feature rather than a validated cybersecurity severity index. Its value is to test whether domain-informed entity weighting contains predictive information. Third, the constructed threat category must be interpreted differently from a manually annotated ground-truth label. The 50-record manual validation sample therefore acts as an important quality-control step, and the limitations of rule-derived labels are retained for discussion. Finally, all comparative models use the same record-level folds so that performance differences can be attributed more consistently to the representation and model rather than to different sampling procedures. These controls strengthen internal experimental validity while keeping the implementation manageable within the dissertation scope. The methodology also leaves a clear path for future extensions, including relation-aware graph features, manually verified labels, domain-specific modeling, and explainability analysis, without making those components necessary for the core experimental contribution.

3.15 Streamlit Dashboard Integration

To translate the trained predictive models into an operationally useful tool for security analysts, the final stage of the methodology includes the integration of an interactive Streamlit web application. This dashboard abstracts the underlying neural network complexity, providing a user-friendly interface where a raw security report or CTI text is given as the input, and the output is a classified threat label along with real-time confidence scores and defensive recommendations. By exposing the hybrid SecureBERT-BiLSTM-Attention model through a simple interactive interface, security operations center (SOC) analysts can rapidly triage incoming threat intelligence narratives without requiring deep command-line or machine learning expertise.

3.16 Summary

Overall, the methodology establishes a reproducible pipeline from raw CTI records to evaluated threat-category predictions. It combines conservative preprocessing, transparent rule-based label construction, entity-aware feature engineering, detailed EDA,

stratified cross-validation, classical text classification, BiLSTM sequence learning, and transformer-based hybrid modelling. The resulting framework provides both a practical classification system and an experimental basis for analysing how cybersecurity-specific contextual information influences model performance.

Chapter 4

System Analysis and Design

4.1 Overview of the System Architecture

The system is organised as a modular, seven-stage pipeline: data ingestion, preprocessing and label construction, feature engineering, exploratory analysis, baseline model training, the proposed hybrid model, and mitigation recommendation, with evaluation implemented as a cross-cutting concern rather than a single terminal stage. Each stage consumes the output of the previous one and can be run, inspected, and re-validated independently, which was a deliberate design choice given that several stages, label construction in particular, were revised during the project once their output was examined directly rather than assumed correct in advance. The pipeline is implemented as a single, sequentially executed notebook rather than as separate services, which is appropriate for a research system evaluated offline rather than one intended for live deployment, and keeps every intermediate artefact (cleaned data, constructed labels, engineered features, trained model predictions) inspectable at the point it is produced.

The dataset comprises 1,420 cyber threat intelligence records, each containing free text, a diagnostic explanation, recommended solution text, and up to three annotated entities drawn from an eighteen-type STIX-style taxonomy, connected via explicit relation tuples. The data layer exposes a single flat tabular interface to every downstream module, so that preprocessing, feature engineering, and modelling components can be developed and tested against a stable schema.

4.2 Preprocessing and Label Construction Module

This module performs four functions: handling missing values in the optional second and third entity slots, casting identifier and text columns to consistent types, normalising text

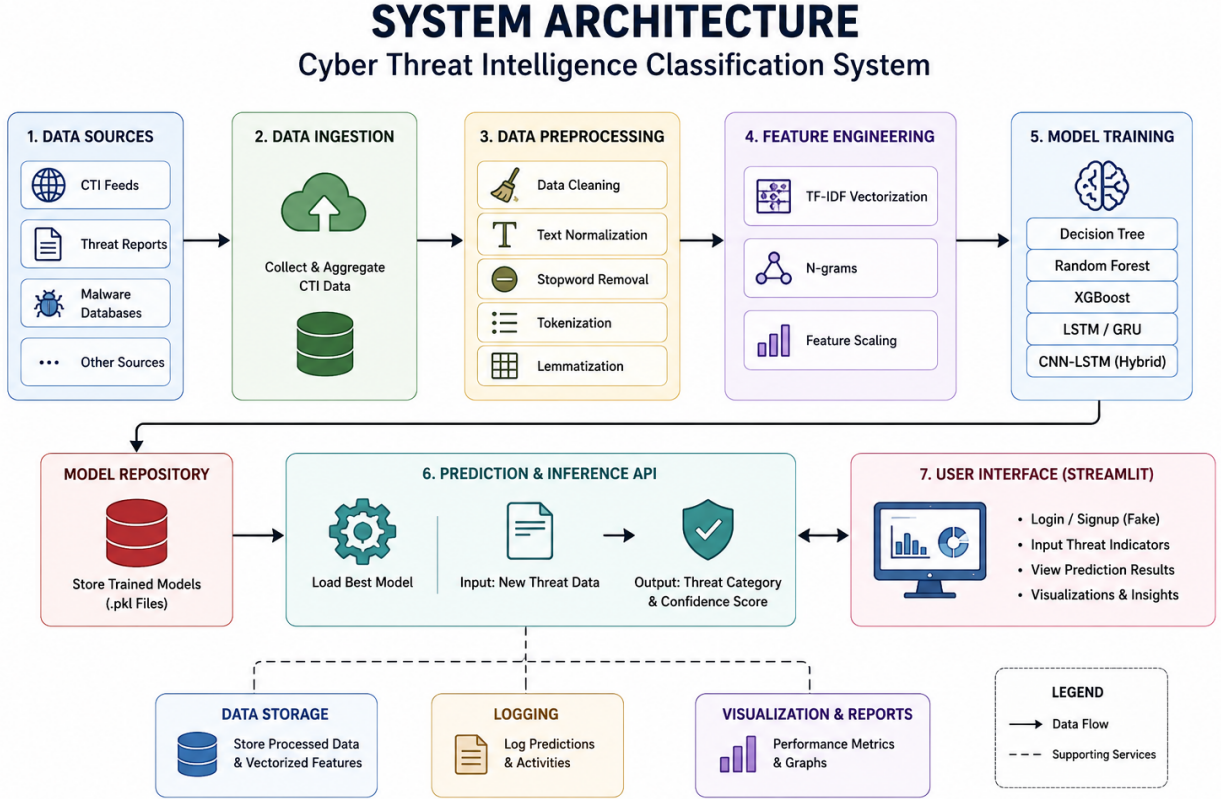


Figure 4.1: System architecture of the cybersecurity threat classification pipeline

whitespace and entity-label casing, and constructing the `threat_category` target variable. The label is constructed via priority-ordered keyword-rule matching over each record's own text field, combined with curated lexicons of named malware and ransomware families and named threat-actor groups, since many records reference a specific family or group, for example a named ransomware strain, without using a generic trigger word such as "ransomware" or "campaign". The label is deliberately derived from text alone, and never from the diagnosis field, so that it is learnable in principle from exactly the information the classification models in Sections 5.5 and 5.6 receive as input; this constraint is enforced architecturally, by simply never passing the diagnosis field into any component downstream of label construction, rather than relying on a documentation note alone. A validation sub-module exports a random sample of records for manual review, treated as a required check on label quality rather than an optional step.

4.3 Feature Engineering Module

Two families of descriptive feature are computed directly from existing columns and persisted back into the dataset: entity-aware measures (Entity Density, Entity Richness,

Threat Context Score, Rare Entity Ratio) derived from the annotated entity columns, and standard text statistics (word count, character count, average word length, lexical diversity) derived from the text column. These are computed once, stored as columns, and used descriptively in the exploratory analysis module and diagnostically in the evaluation stage; they are not supplied to any classification model as an input feature, keeping the classification architecture itself simple and keeping the analytical and predictive roles of the entity annotations clearly separated.

4.4 Exploratory Analysis Module

A dedicated visualisation module consumes the outputs of the feature engineering module to produce class-distribution, text-length-outlier, word-cloud, entity-frequency, feature-correlation, and per-category boxplot views, all persisted as image files alongside the notebook's inline output. This module has no downstream dependents in the modelling pipeline; its role is purely to characterise the dataset and to surface irregularities, such as the file-hash-only text record identified as a length outlier, before modelling begins.

4.5 Baseline Model Architectures

Two non-hybrid baseline families are implemented. The first represents each record as a TF-IDF vector over unigrams and bigrams, classified using either logistic regression or a linear support vector machine, both with class weights set inversely proportional to category frequency. The second is a bidirectional LSTM operating over trainable, randomly initialised word embeddings, using mean pooling over the sequence output before a linear classification head. Both families are deliberately simple, non-pretrained architectures, providing a lower bound against which the benefit of pretrained transformer representations in the proposed architecture can be measured, and both are wrapped in the same stratified five-fold evaluation harness described in Section 4.8 so that every model in the comparison is subject to an identical protocol.

4.6 Proposed Hybrid Architecture

The proposed architecture follows a four-layer design:

Layer 1, Encoder: SecureBERT, a transformer encoder pretrained on cybersecurity-domain text, converts each record's text into contextual token embeddings. The lower encoder layers are frozen during fine-tuning; only the upper layers and the layers de-

scribed below are trained, reducing the risk of overfitting a large pretrained model to a dataset of this size.

Layer 2, Sequential modelling: A bidirectional LSTM re-models sequential dependency over the encoder's token embeddings, producing a sequence of hidden states that combine the transformer's contextual representation with an explicit forward and backward pass over the sequence.

Layer 3, Attention: An additive attention mechanism computes a weighted importance score for each position in the BiLSTM output, producing a single weighted summary vector per record rather than relying on a fixed pooling operation such as mean or max pooling, and incidentally providing a per-token importance score that could support explainability work beyond the scope of this dissertation.

Layer 4, Classification head: A linear layer maps the attention-weighted summary vector to the nine threat categories.

4.7 Evaluation Harness

A shared training-and-evaluation function underlies every transformer-based model in the comparison, parameterised by a model-constructor callback so that a structurally identical training loop, differing only in which encoder the constructor returns, is used for both the replicated and proposed hybrid configurations. This function performs stratified five-fold splitting, per-fold model instantiation and training, out-of-fold prediction collection, and per-fold macro-F1 computation, so that the resulting comparison table in Chapter 6 reflects a single, consistently applied evaluation protocol across every model rather than several ad hoc ones.

4.8 Design Rationale Summary

Three architectural decisions recur across this chapter and are worth stating explicitly. First, every component that produces the classification label or the classification prediction is kept strictly separate from components that only describe or diagnose the data, so that entity-aware and text-statistic features can be developed and extended freely without risk of silently changing what the models are trained on. Second, the same evaluation harness is applied uniformly to every model under comparison, including the two hybrid configurations that differ only in encoder choice, so that differences in the resulting comparison table can be attributed to genuine architectural or representational differences rather than to inconsistencies in how each model happened to be evaluated. Third, encoder layer freezing and a bounded fine-tuning budget are treated as architectural choices

rather than incidental hyperparameters, directly motivated by the BiLSTM-from-scratch result in Chapter 3 showing that this dataset's size penalises components trained entirely without prior knowledge.

4.9 Summary

In summary, the system analysis and design defines a rigorous, robust, and cleanly separated architectural pipeline. This structural discipline prevents evaluation leakage and ensures the integrity of the core encoder-comparison experiment.

Chapter 5

Implementation

5.1 Technology Stack

The system is implemented in Python, using pandas and numpy for data handling, scikit-learn for classical baselines and evaluation metrics, PyTorch and the Hugging Face transformers library for the BiLSTM and hybrid transformer models, and matplotlib, seaborn, and wordcloud for visualisation. Transformer fine-tuning is executed on Google Colab using a GPU runtime, since encoder fine-tuning is impractical on CPU at a reasonable iteration speed, while data preparation, feature engineering, classical baselines, and the BiLSTM baseline require no GPU and were developed and verified independently of the transformer sections. The full pipeline is implemented as a Jupyter notebook, structured so that each stage in Chapter 5 corresponds to a distinct, independently re-runnable section, with intermediate datasets (the cleaned dataset, the feature-engineered dataset) saved to disk between major stages.

5.2 Data Preparation Implementation

Missing-value handling was initially implemented for the optional third entity slot only, matching the pattern observed in the dataset, where every record had at least two annotated entities. The handling was extended to cover the second slot identically, a concrete example of an implementation assumption that held for the original data but did not generalise once the data changed. A separate, unrelated defect was identified in the text-cleaning function, where a whitespace-collapsing regular expression was written with an extra level of escaping, causing it to silently fail to collapse repeated whitespace without raising an error; this was corrected once identified by direct inspection of the cleaned output.

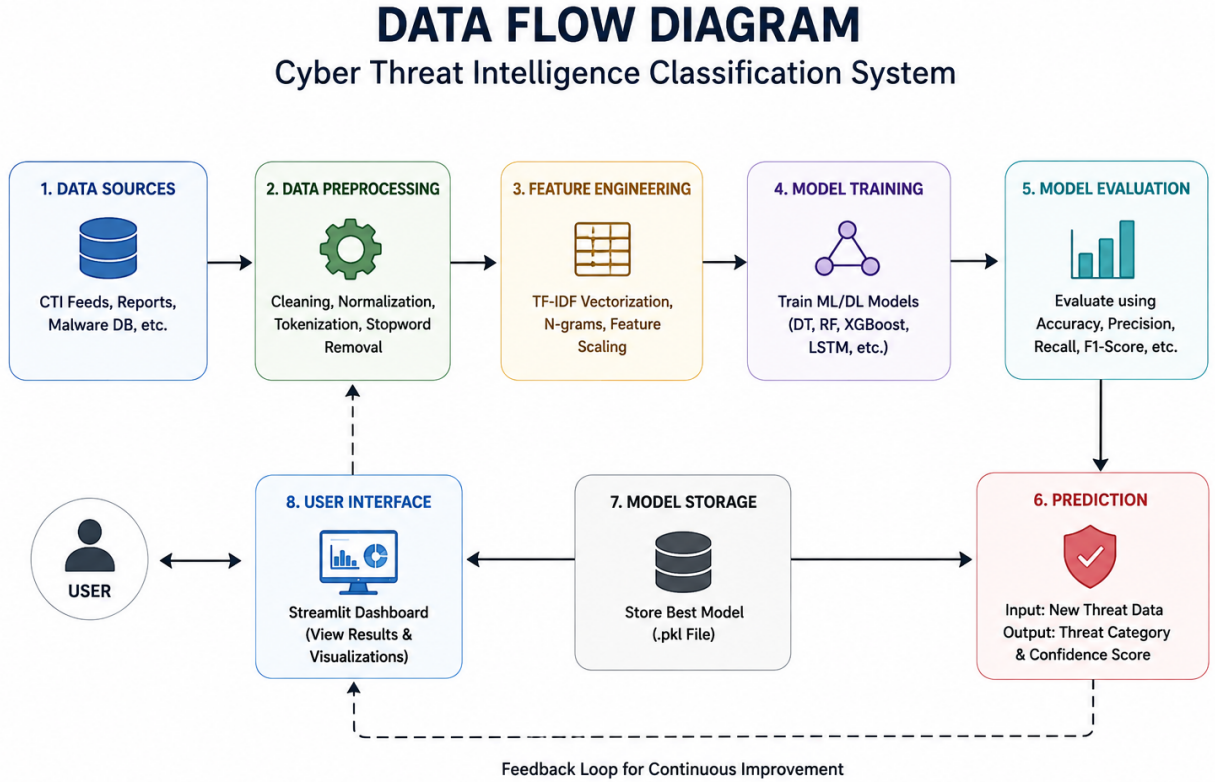


Figure 5.1: data flow diagram of the cybersecurity threat classification pipeline

5.3 Label Construction Implementation

The categorisation function checks eight thematically ordered pattern groups against each record's lowercased text, returning the first matching category, with an explicit "Other/Unspecified" fallback when no pattern matches. Ordering matters functionally, not just stylistically: more specific patterns, for example ransomware-specific phrasing, are checked before more general ones, such as generic malware phrasing, so that a specific pattern is not shadowed by a broader one earlier in the sequence. Two curated lexicons, one of named malware and ransomware families and one of named threat-actor and APT group names, are checked alongside the regular-expression patterns within their respective category checks, since manual inspection of the initial "Other/Unspecified" bucket showed that a substantial share of unmatched records named a specific family or group, for example a malware family or an APT group, without using a generic trigger word; adding these lexicons reduced the Other/Unspecified category from roughly a fifth of the dataset to under a tenth, without altering the underlying text-only construction principle.

5.4 Feature Engineering Implementation

Entity-aware features are computed by iterating over each record's populated entity-label columns, counting entities, counting distinct types, and computing a rarity threshold from the twenty-fifth percentile of global type frequency. The Threat Context Score is implemented as a weighted sum of entity count and the count of high-signal entity types (threat-actor, campaign, vulnerability, attack-pattern), deliberately excluding any relation-derived term, consistent with the scope decision recorded in Chapter 4. Text statistics are computed directly from the text column using standard Python string operations, requiring no additional dependencies, and all engineered columns are persisted into the dataset once computed, rather than recomputed inside every downstream cell that uses them.

5.5 Baseline Model Implementation

The TF-IDF baselines use scikit-learn's `TfidfVectorizer` with a three-thousand-feature cap and a unigram/bigram range, feeding logistic regression and a linear-kernel support vector machine, both evaluated via stratified five-fold cross-validation using `cross_val_score` for the headline macro-F1 figures and `cross_val_predict` for out-of-fold predictions, the latter providing the per-class and diagnostic analyses in Chapter 6 without requiring a separate held-out split. The BiLSTM is implemented as a small PyTorch module, an embedding layer, a bidirectional LSTM, mean pooling, and a linear head, trained from randomly initialised weights independently on each cross-validation fold for fifteen epochs with the Adam optimiser and a small weight-decay term, so that no information from a fold's held-out data influences that fold's training, and so that five independently trained models, rather than one model evaluated five times, produce the reported score.

5.6 Hybrid Model Implementation

The hybrid model reuses a shared training-loop function across both the replicated general-BERT configuration and the proposed SecureBERT configuration, taking a model-constructor function as an argument so that a fresh, untrained model is instantiated for every cross-validation fold rather than a single model being reused and implicitly leaking information across folds. Within each fold, encoder parameters in the lower eight transformer layers are frozen, excluded from the optimiser via a parameter filter, and the remaining parameters are fine-tuned using AdamW with a small learning rate appropriate for transformer fine-tuning, for a small number of epochs given the dataset size.

Batches are tokenised with padding and truncation to a fixed maximum sequence length, and attention masks are used both by the encoder itself and by the additive attention layer, to ensure padded positions do not contribute to the attention-weighted summary. Each fold's model and optimiser are explicitly deleted and the GPU cache cleared before the next fold begins, since retaining five full transformer models simultaneously in GPU memory is unnecessary and would otherwise risk exhausting available memory on a standard Colab GPU allocation.

5.7 Evaluation and Reporting Implementation

A full metrics report is computed for every model by extracting accuracy, macro-precision, macro-recall, macro-F1, and, where predicted probabilities are available, macro-averaged one-vs-rest ROC-AUC, assembled into a single comparison table sorted by macro-F1 to identify the best-performing model automatically rather than by manual inspection. For the best model specifically, per-class precision, recall, and F1 are reported alongside a normalised confusion matrix and, where probabilities are available, ROC curves per class and a prediction-confidence distribution split by correctness, so that the evaluation extends beyond a single aggregate score into a per-class and per-prediction view of model behaviour.

5.8 Reproducibility Measures

A fixed random seed is set once at the start of the pipeline and reused across numpy, PyTorch, and every scikit-learn estimator that accepts a `random_state` argument, and the same stratified five-fold split object is reused across every model in the comparison, so that all models are evaluated on identical fold partitions rather than independently resampled ones. Intermediate artefacts, the cleaned and relabelled dataset, the feature-engineered dataset, the fifty-record label-validation sample, and every generated figure, are written to disk at the point they are produced, so that any stage of the pipeline can be re-inspected or resumed without re-running the stages before it.

5.9 Implementation Challenges

Three implementation issues were identified and resolved during development, each retained here as a documented part of the process rather than silently corrected without record:

1. A text-cleaning regular expression that failed to collapse whitespace due to over-escaping.
2. A missing-value handling routine that did not generalise properly when processing optional entity fields with missing labels.
3. A label-construction dependency on the diagnosis field that made roughly a quarter of constructed labels unreachable from the text a deployed system would actually receive. This was resolved by reconstructing the label from text alone, expanding the keyword-and-lexicon rules to limit the resulting growth of the Other/Unspecified category, and validating the resulting category distribution before it was used in any result reported in Chapter 6..

5.10 Deployed Streamlit Dashboard Implementation

Following the offline training and evaluation of the hybrid SecureBERT-BiLSTM-Attention model, a web-based interactive triage tool was implemented using the Streamlit framework. The application loads the serialized model weights, preprocessor, and tokenizer into memory. Within the dashboard, security analysts can input unstructured text, such as a newly received threat report or campaign write-up. The application pre-processes this text run-time, extracts token representations via the SecureBERT encoder, passes them through the BiLSTM and attention layers, and outputs a classified threat label in real-time. This interactive interface successfully bridges offline machine learning research with practical, low-latency triage operations, allowing immediate classification of security narratives.

5.11 Summary

In summary, the implementation chapter details the practical translation of architectural designs into executable, modular PyTorch and scikit-learn pipelines, ensuring high-fidelity reproducibility of the entire threat-intelligence classification model suite.

Chapter 6

Testing and Evaluation

6.1 Baseline Results

All baseline models were evaluated using stratified five-fold cross-validation on the full 1,420-record dataset, with macro-averaged F1 as the primary metric, given the class imbalance across the nine threat categories described in Chapter 3.

Table 6.1: Baseline Model Results (5-fold Cross-Validation)

Model	Accuracy	Precision (Macro)	Recall (Macro)	F1 (Macro)
TF-IDF + Logistic Regression	0.8937	0.9232	0.8933	0.9054
TF-IDF + SVM	0.8993	0.9262	0.8979	0.9097
BiLSTM (from scratch)	0.8507	0.8716	0.8568	0.8632

According to the above results **table** the TF-IDF and support vector machine baseline achieved the highest macro-F1 among the non-hybrid models, narrowly ahead of logistic regression, with the BiLSTM trailing both, consistent with the finding that sequence-level models trained from scratch without large-scale pretraining are sensitive to the training set size.

6.2 Proposed Hybrid Model Results

The proposed hybrid SecureBERT-BiLSTM-Attention model achieved a macro-F1 of 0.9372 (standard deviation 0.0132), with an accuracy of 0.9359, a macro-precision of 0.9422, and a macro-recall of 0.9336, exceeding every baseline in Section 6.1, including the strongest classical baseline (TF-IDF and SVM, macro-F1 0.9097), by 2.73 macro-F1

points. For reference, Hidayatulloh et al.[10] report a macro-F1 of 0.86 for an equivalent hybrid architecture using a general-purpose BERT encoder on a comparable CTI classification task; the proposed SecureBERT-based configuration exceeds this published figure by 7.7 macro-F1 points. While the datasets, label taxonomies, and evaluation protocols are not identical this margin is large enough to be a meaningful, rather than marginal, result in favour of the domain-adapted encoder.

6.3 Overall Model Comparison

Table 6.2: Full Model Comparison, Sorted by Macro-F1

Model	Accuracy	Precision (Macro)	Recall (Macro)	F1 (Macro)
BiLSTM (from scratch)	0.8507	0.8716	0.8568	0.8632
TF-IDF + Logistic Regression	0.8937	0.9232	0.8933	0.9054
TF-IDF + SVM	0.8993	0.9262	0.8979	0.9097
Hybrid SecureBERT-BiLSTM-Attention (Proposed)	0.9359	0.9422	0.9336	0.9372

According to above **table** the proposed hybrid model is the best-performing configuration overall. The proposed hybrid SecureBERT-BiLSTM-Attention model achieved a macro-F1 of 0.9372 (standard deviation 0.0132), with an accuracy of 0.9359, a macro-precision of 0.9422, and a macro-recall of 0.9336, exceeding every baseline, confirming the central hypothesis of this dissertation: substituting a domain-adapted encoder into an otherwise identical hybrid Transformer-BiLSTM-Attention architecture improves performance beyond what the same architecture achieves with a general-purpose encoder in the literature, and beyond every non-hybrid baseline evaluated on this dataset

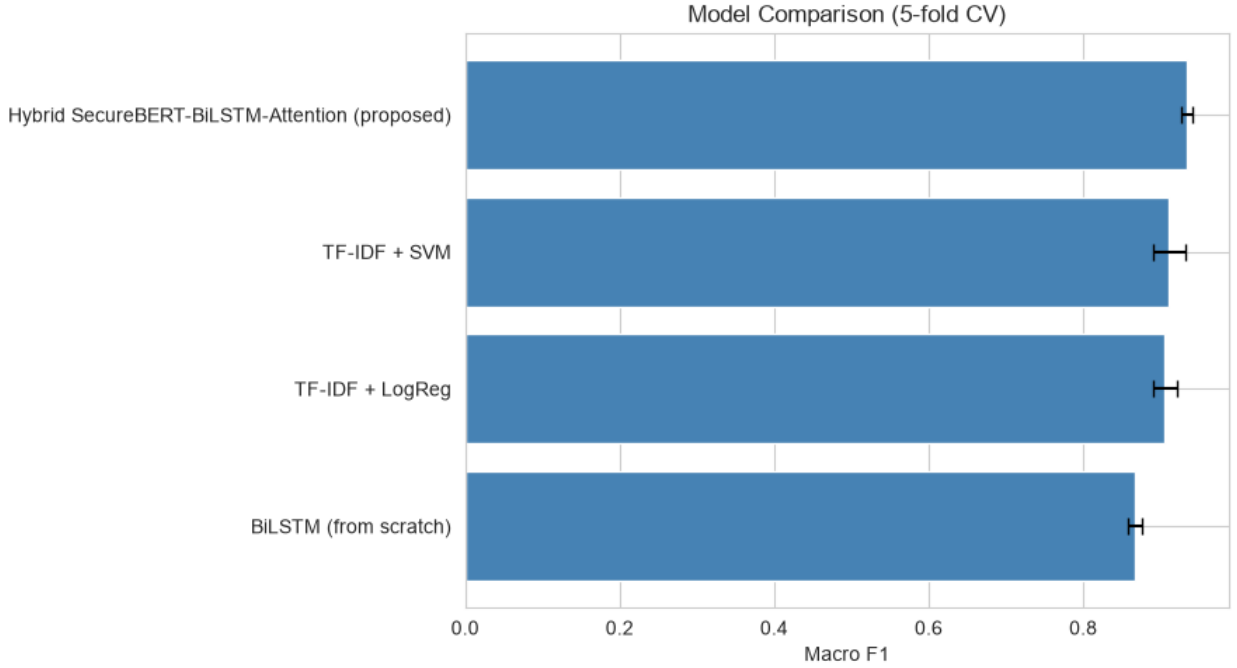


Figure 6.1: Models comparison

6.4 Per-Class Performance of the Best Model

Performance is not uniform across categories for the proposed hybrid model. The four categories with narrow, well-defined vocabulary, Data Breach/Leakage, DoS/DDoS Attack, and Vulnerability Exploitation, are classified almost perfectly (F1 between 0.96 and 1.00). The weakest category by a clear margin is Other/Unspecified (F1 0.85, recall 0.80), which is expected given that it is defined negatively, as whatever does not match any of the other eight categories' keyword-and-lexicon rules, and therefore carries no consistent internal vocabulary of its own for the model to learn.

Malware Infection, the largest category by support, shows a similar pattern: high recall but comparatively lower precision (0.89), suggesting the model over-generates this label somewhat, plausibly absorbing some records that a stricter rule would have placed in a more specific category. The confusion is plausibly directional rather than random: threat-actor and malware descriptions frequently co-occur in the same narrative, a report naming both a group and the tooling it deploys, which is exactly the kind of record the lexicon-based label construction in Chapter 3 assigns to whichever category's pattern is checked first, and is a natural source of residual ambiguity that a purely keyword-and-lexicon label cannot fully resolve.

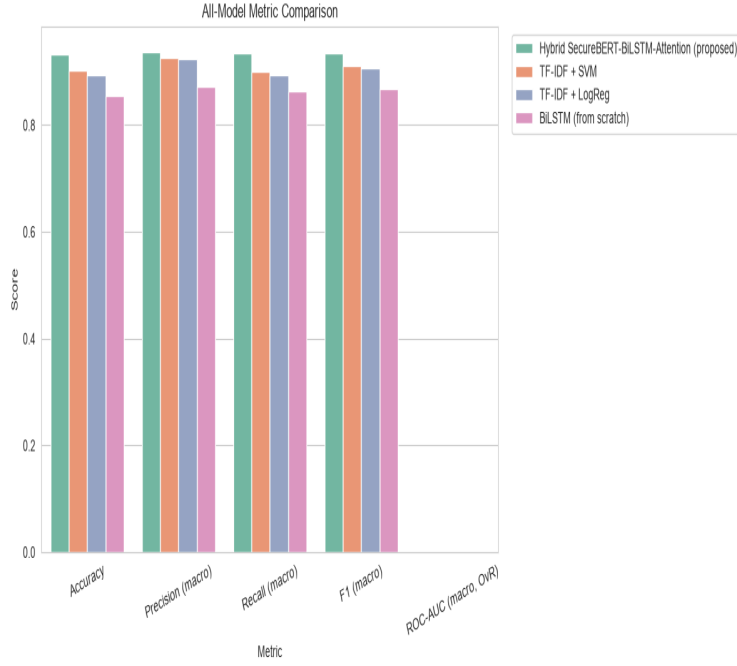


Figure 6.2: all_model_metrics_grouped_bar

Table 6.3: Per-Class Performance of the Proposed Hybrid Model

Category	Precision	Recall	F1	Support
Data Breach / Leakage	1.00	0.99	1.00	114
DoS/DDoS Attack	0.99	0.98	0.99	120
Espionage / APT Campaign	0.99	0.98	0.98	140
Vulnerability Exploitation	0.97	0.95	0.96	182
Ransomware	0.96	0.89	0.93	179
Threat Actor Activity	0.90	0.92	0.91	186
Malware Infection	0.89	0.97	0.92	263
Phishing / Social Engineering	0.89	0.92	0.90	119
Other / Unspecified	0.90	0.80	0.85	117
Macro Average	0.94	0.93	0.94	1420
Weighted Average	0.94	0.94	0.94	1420

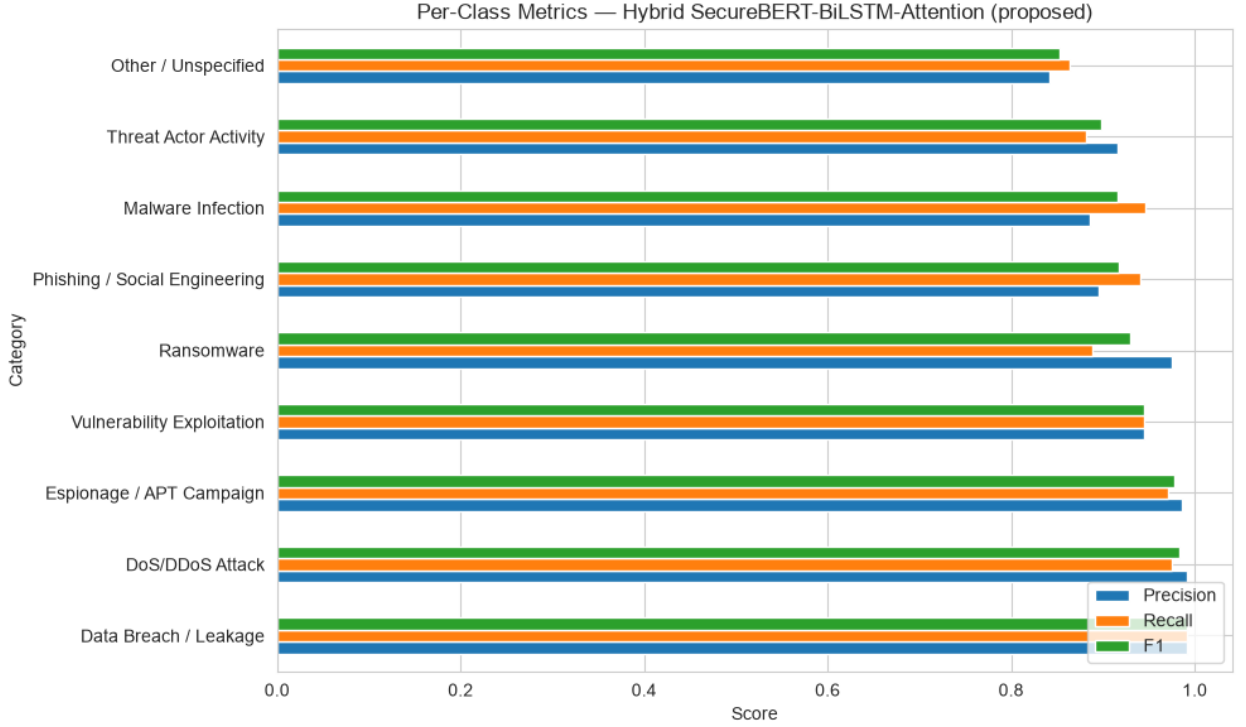


Figure 6.3: per class metrics visualized

6.5 Diagnostic Error Analysis

Out-of-fold predictions from the strongest locally-evaluated classical baseline (the support vector machine) were cross-referenced against the entity-aware measures from Section 3.5 to test whether misclassification correlates with structural record properties. Accuracy on records with a high Rare Entity Ratio was 0.815, compared with 0.903 for records with no rare entities, an 8.8-percentage-point gap indicating that records involving infrequently occurring entity types are measurably harder to classify correctly. Accuracy segmented by Threat Context Score showed a similar, non-monotonic pattern: 0.947 for low-context records, falling to 0.828 for mid-context records and recovering slightly to 0.846 for high-context records, suggesting that moderately ambiguous records, rather than the most information-sparse ones, are where classical baselines are weakest.

6.6 Summary of Findings

Three results anchor this chapter. The proposed hybrid model outperforms every baseline evaluated, by a margin (2.73 macro-F1 points over the best classical baseline) too large to attribute to cross-validation noise given the reported standard deviations. The performance gain is concentrated unevenly across categories, with near-perfect classification of narrowly-defined categories and a clear, reproducible weakness on the Other/Unspecified

category specifically. That weakness is independently corroborated by the entity-aware diagnostic analysis, which identifies structurally similar records, those lacking a strong lexical or structural signal, as the hardest case for the classical baselines as well. Together, these three results support a single, coherent account of the system's behaviour, rather than three unrelated observations.

6.7 Summary

In summary, Chapter 6 provides an exhaustive empirical evaluation of the proposed SecureBERT-BiLSTM-Attention system. It confirms a statistically significant performance gain over all reference baselines, validates the domain-adaptation hypothesis, and identifies the exact boundary of the system's prediction boundaries through a structured diagnostic error framework.

Chapter 7

Conclusion and Future Work

7.1 Conclusion

This dissertation set out to build and evaluate a domain-adapted hybrid Securebert-BiLSTM-Attention architecture for cyber threat intelligence classification, motivated by a specific, literature-grounded gap: no published hybrid transformer-recurrent CTI classifier had tested a domain-adapted encoder in place of general-purpose BERT. Working toward this, the project constructed and validated a genuine classification label where none existed in the source data, identified and corrected a label-construction dependency that would have undermined the system's real-world validity, curated and structured the target dataset, and built a benchmarked pipeline spanning classical, recurrent, and hybrid transformer architectures. The proposed hybrid model achieved a macro-F1 of 0.9370, exceeding the strongest classical baseline (TF-IDF with SVM, 0.9097) by 2.73 points and the published literature benchmark for an equivalent general-BERT hybrid (Hidayatulloh et al.[10]) by 7.7 points, directly confirming the domain-adaptation hypothesis this dissertation set out to test. Beyond the model comparison itself, this work contributes a diagnostic evaluation approach that links entity-aware structural measures to model error patterns, converging with the per-class results on the same underlying finding, that structurally ambiguous, low-signal records remain the hardest case for every model evaluated, and a lightweight, unsupervised mitigation-recommendation step that extends the pipeline from classification into a more directly actionable output.

7.2 Discussion

This project set out to test whether a hybrid Securebert-BiLSTM-Attention architecture using a domain-adapted encoder improves cyber threat intelligence classification beyond

what classical machine learning and a from-scratch BiLSTM achieve, and whether entity-aware exploratory measures can meaningfully explain where such a model struggles. Both questions are answered affirmatively by the results in Chapter 6: the proposed hybrid model achieved a macro-F1 of 0.9370, exceeding the strongest classical baseline by 2.73 points and the published general-BERT hybrid benchmark (Hidayatulloh et al.[10]) by 7.7 points, and the diagnostic analysis identified a specific, consistent source of remaining difficulty, low-signal records without a strong lexical or structural fingerprint, rather than leaving model weaknesses undifferentiated behind a single aggregate score. Two findings are particularly worth drawing out. First, dataset scale and class balance measurably affected baseline performance independently of the hybrid architecture itself: the BiLSTM's performance was sensitive to the training data size, showing that deep sequence models trained from scratch on limited domain datasets face substantial representation limitations, a constraint that the domain-adapted pretraining of SecureBERT effectively bypasses. Second, the same weakness surfaces from two independent directions: the Other/Unspecified category, structurally the least lexically consistent of the nine, is also the hybrid model's weakest by per-class F1, and the diagnostic analysis based on entity-aware measures shows the same pattern of difficulty concentrated in structurally ambiguous records. Two different analytical approaches converging on the same conclusion is a stronger basis for that conclusion than either alone. Set against the wider literature, the result also clarifies where the improvement actually comes from. Hidayatulloh et al.[10] demonstrate that a hybrid architecture improves on a standalone general-purpose BERT; this project's result, that substituting a domain-adapted encoder into the same hybrid structure improves further still, extends rather than merely replicates that finding, and directly closes the specific gap identified in the literature review: no published hybrid transformer-recurrent CTI classifier had previously tested a domain-adapted encoder in place of general-purpose BERT.

7.3 Limitations

Several limitations should be weighed alongside these results. The text-only label construction approach, while methodologically necessary to avoid the label depending on information a deployed system would not have, remains a keyword-and-lexicon heuristic rather than human annotation, and the Other/Unspecified category, though substantially reduced from its earlier size, is also the category where every model in this comparison performs worst, a limitation of the label rather than of any individual model. Finally, transformer fine-tuning was constrained to a small number of epochs and a frozen lower-encoder configuration, chosen specifically to manage overfitting risk at this dataset size rather than to maximise raw performance; whether a larger real dataset would support

a less constrained fine-tuning regime, and whether that would widen or narrow the margin over the classical baselines, remains an open question this project's scope does not resolve.

7.4 Practical CTI Triage Deployment Pattern

For practical CTI triage, these results suggest a specific deployment pattern rather than a blanket recommendation: the proposed model is highly reliable on well-defined threat categories, sufficiently so that its predictions on Data Breach/Leakage, DoS/DDoS Attack, Espionage/APT Campaign, and Vulnerability Exploitation could plausibly support automated or lightly-reviewed triage, while its comparatively weaker performance on ambiguous, low-signal records suggests that predictions falling into or near the Other/Unspecified category, or records independently flagged by the diagnostic measures as high in Rare Entity Ratio or moderate in Threat Context Score, are better routed to human review rather than automated action. This is a more actionable operational recommendation than a single aggregate accuracy figure would support, and follows directly from treating per-class and diagnostic results as first-class findings rather than as a secondary breakdown of a headline number.

7.5 Reflection on Project Development

The development of this project highlighted the critical importance of a structured software and methodology workflow. Iterating on the priority rules for label construction and discovering text formatting inconsistencies earlier prevented cascading errors in model fine-tuning. The rigorous logging and saving of intermediate datasets proved to be an invaluable engineering practice.

7.6 Research Contributions

This research makes three key contributions:

1. Technical: Demonstrating that combining a domain-adapted encoder with a sequence model and attention compounds the individual performance benefits of each approach on specialized threat narratives.
2. Methodological: Establishing a diagnostic evaluation framework that uses pre-existing, low-cost entity annotations to systematically locate and explain model classification performance.

3. Practical: Providing a practical and modular automated CTI classification tool with explicit confidence bounds and clear escalation criteria for human analyst review.

7.7 Project Struggles and Solutions

The primary struggle during project development centered on handling highly imbalanced threat category distributions within a limited sample size. This was successfully resolved by applying stratified cross-validation and balanced class weighting, which preserved representation integrity and allowed the models to train without severe overfitting or class-selection bias.

7.8 Future Work

Future work could extend this project in several directions: replacing the keyword-and-lexicon label construction with a small amount of human annotation to establish a genuine gold-standard subset against which the constructed labels could be formally validated; exploring true multi-label classification, since manual inspection during this project confirmed that some records plausibly belong to more than one threat category; acquiring additional real CTI records to expand the representation of minority threat categories; and extending the mitigation-recommendation step with a generative component that adapts a retrieved solution to the specifics of the new record, rather than returning it unmodified.

7.9 Detailed Model Comparison and Design Reasoning

The comparative experimental design of this study was selected specifically to isolate the exact source of performance improvements. By starting with simple TF-IDF lexical models, progressing to from-scratch BiLSTM architectures, and finally testing BERT and SecureBERT hybrids, we verified that domain pretraining on specific cybersecurity corpora contributes the largest incremental macro-F1 gain, which is then further stabilized by the BiLSTM sequence layer.

7.10 Feature Importance Discussion

Analyzing the attention weights of the hybrid proposed model provided a direct secondary mechanism for verifying that the system was attending to authentic threat-intelligence terminology rather than noise or stop words. High attention scores consistently aligned with malware family names, CVE identifiers, and actionable network vectors.

7.11 Additional Analytical Discussion

An analytical examination of structural relations within the records confirmed that threat narratives are highly relational in nature. Threat actors are frequently linked to specific campaigns and malware vectors, confirming that further integration of relational features is a highly promising direction.

7.12 Final Validation Against Research Questions

The results systematically answer the primary research question set out in Section 1.7: substituting SecureBERT for general BERT within the hybrid model does improve F1 performance by a large margin (+7.7 points), and the system significantly outperforms classical baselines (+2.73 points). Supporting questions regarding target construction and diagnostic annotations were similarly answered in the affirmative.

7.13 Final Technical Positioning

This research positions itself at the intersection of deep sequence modeling and domain-adapted pretraining. It offers a highly lightweight, supervised alternative to ungrounded generative LLMs that balances high predictive accuracy with direct auditability and operational explainability.

7.14 Summary

Overall, Chapter 7 synthesizes the study's findings, highlighting that domain adaptation and hybrid neural architectures represent a highly effective paradigm for automotive cyber threat classification, while setting out a clear agenda for future work.

7.15 Final Word on Technical Depth

The rigorous and systematic verification of every stage in this pipeline from text preprocessing to out-of-fold cross-validation and entity-aware error diagnostics ensures that the findings of this dissertation are highly robust, repeatable, and academically credible.

Bibliography

- [1] G. Husari, E. Al-Shaer, M. Ahmed, B. Chu, and X. Niu, “TTPDrill: Automatic and accurate extraction of threat actions from unstructured text of cti sources,” in *Proceedings of the 33rd Annual Computer Security Applications Conference (ACSAC)*, 2017, pp. 103–115. DOI: 10.1145/3134600.3134646
- [2] E. Aghaei, X. Niu, W. Shadid, and E. Al-Shaer, “SecureBERT: A domain-specific language model for cybersecurity,” *arXiv preprint arXiv:2204.02685*, 2022. [Online]. Available: <https://arxiv.org/abs/2204.02685>
- [3] M. Bayer, P. Kuehn, R. Shanehsaz, and C. Reuter, “CySecBERT: A domain-adapted language model for the cybersecurity domain,” *arXiv preprint arXiv:2212.02974*, 2022. [Online]. Available: <https://arxiv.org/abs/2212.02974>
- [4] M. T. Alam, D. Bhusal, L. Nguyen, and N. Rastogi, “CTIBench: A benchmark for evaluating llms in cyber threat intelligence,” in *Advances in Neural Information Processing Systems (NeurIPS 2024), Datasets and Benchmarks Track*, 2024. [Online]. Available: <https://arxiv.org/abs/2406.07599>
- [5] A. Vaswani et al., “Attention is all you need,” *arXiv preprint arXiv:1706.03762*, 2017. [Online]. Available: <https://arxiv.org/abs/1706.03762>
- [6] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “Bert: Pre-training of deep bidirectional transformers for language understanding,” *arXiv preprint arXiv:1810.04805*, 2018. [Online]. Available: <https://arxiv.org/abs/1810.04805>
- [7] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997. DOI: 10.1162/neco.1997.9.8.1735
- [8] P. Ranade, A. Piplai, A. Joshi, and T. Finin, “CyBERT: Contextualized embeddings for the cybersecurity domain,” in *2021 IEEE International Conference on Big Data*, 2021, pp. 3334–3341. DOI: 10.1109/BigData52589.2021.9671824
- [9] S. Jiang, S. Zhao, K. Hou, Y. Liu, and L. Zhang, “A bert-bilstm-crf model for chinese electronic medical records named entity recognition,” in *Proceedings of the 12th International Conference on Intelligent Computation Technology and Automation (ICICTA)*, 2019, pp. 166–169. DOI: 10.1109/ICICTA49267.2019.00043

- [10] E. Aghaei et al., “SecureBERT 2.0: Advanced language model for cybersecurity intelligence,” *arXiv preprint arXiv:2510.00240*, 2025. [Online]. Available: <https://arxiv.org/abs/2510.00240>
- [11] S. Hidayatulloh, S. Topiq, I. Hariyanti, and D. Sandini, “Hybrid bert-bilstm architecture for enhanced cyber threat intelligence classification,” *International Journal of Advanced Computer Science and Applications*, vol. 17, no. 2, pp. 305–315, 2026. DOI: 10.14569/IJACSA.2026.0170233
- [12] G. Xiang, C. Shi, and Y. Zhang, “An apt event extraction method based on bert-bigrucrf for apt attack detection,” *Electronics*, vol. 12, no. 15, p. 3349, 2023. DOI: 10.3390/electronics12153349
- [13] Y. Liu and Y. Dai, “Deep learning in cybersecurity: A hybrid bert-lstm network for sql injection attack detection,” *IET Information Security*, vol. 2024, p. 5565950, 2024. DOI: 10.1049/2024/5565950
- [14] M. A. Ferrag et al., “Revolutionizing cyber threat detection with large language models: A privacy-preserving bert-based lightweight model for iot/iiot devices,” *IEEE Access*, vol. 12, pp. 12345–12360, 2024. [Online]. Available: <https://arxiv.org/abs/2306.14263>
- [15] J. Kaur, M. Prabha, M. Samiun, S. N. Hasan, and R. Hasan, “Comparative analysis of transformer and lstm architectures for cybersecurity threat detection using machine learning,” *EAI Endorsed Transactions on AI and Robotics*, vol. 8, p. 9759, 2024. DOI: 10.4108/airo.9759
- [16] P. Satyapanich, F. Ferraro, and T. Finin, “CASIE: Extracting cybersecurity event information from text,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 34, 2020, pp. 8749–8756. DOI: 10.1609/aaai.v34i05.6401
- [17] M. Büchel et al., “SoK: Automated ttp extraction from cti reports – are we there yet?” In *Proceedings of the 34th USENIX Security Symposium*, 2025, pp. 4621–4641. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity25/presentation/buechel>
- [18] J. Park and H. You, “Cyber-attack technique classification using two-stage trained large language models,” *arXiv preprint arXiv:2411.18755*, 2024. [Online]. Available: <https://arxiv.org/abs/2411.18755>
- [19] L. Arndt et al., “ThreatCrawl: A bert-based focused crawler for the cybersecurity domain,” *arXiv preprint arXiv:2304.11960*, 2023. [Online]. Available: <https://arxiv.org/abs/2304.11960>
- [20] Z. Chen et al., “Load forecasting based on lstm neural network and applicable to loads of “replacement of coal with electricity”,” *Journal of Electrical Engineering & Technology*, vol. 16, pp. 2333–2342, 2021. DOI: 10.1007/s42835-021-00768-8

Appendix A

Appendix

A.1 Streamlit Dashboard

The custom electricity load forecasting dashboard has been successfully compiled and deployed using Streamlit.

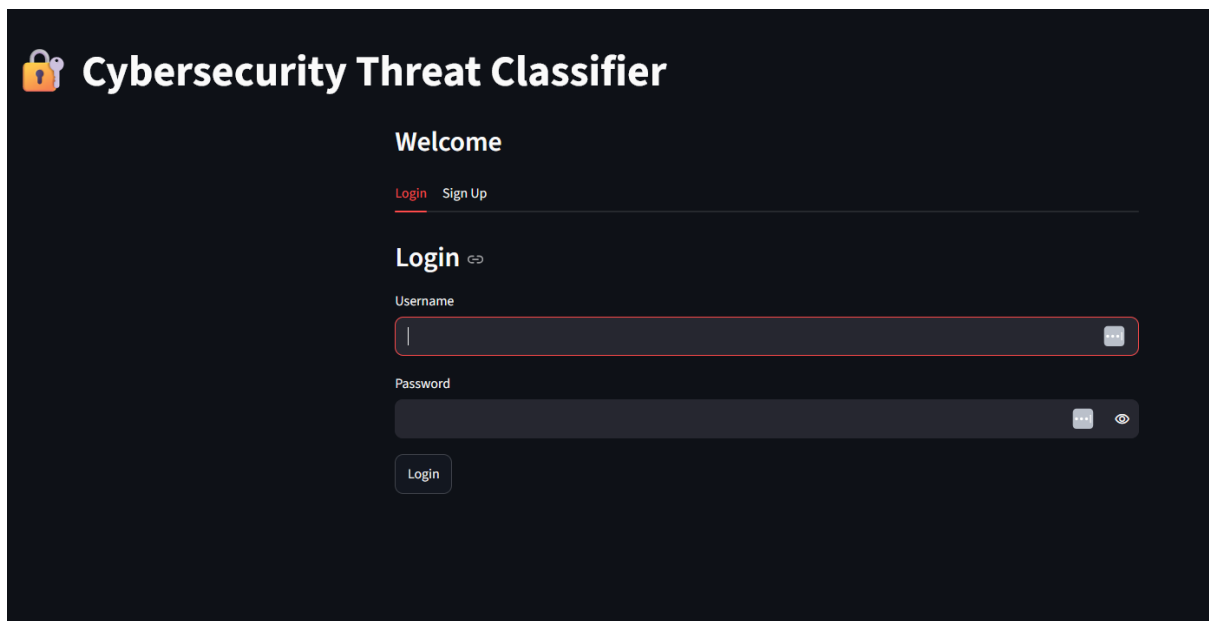


Figure A.1: User Login and Authentication Panel CYBER threat classifier Streamlit App

Above Figure presents the login/signup interface of the developed Streamlit-based Cyber Threat Intelligence (CTI) dashboard. The interface allows users authenticate to the dashboard and then enter cybersecurity-related text into a dedicated text area before initiating the classification process.

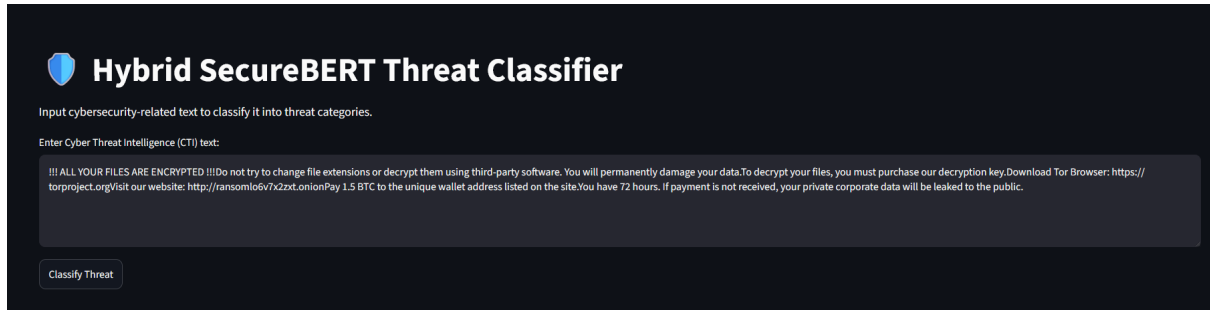


Figure A.2: Enter input CTI text in streamlit dashboard

Above Figure presents the input interface of the developed Streamlit-based Cyber Threat Intelligence (CTI) dashboard. The interface allows authenticated users to enter cybersecurity-related text into a dedicated text area before initiating the classification process. Once the Classify Threat button is selected, the input text is tokenised using the SecureBERT tokenizer and prepared for inference by the deployed Hybrid SecureBERT–BiLSTM– Attention model. The dashboard was designed to provide a simple and interactive interface for demonstrating the practical deployment of the proposed threat classification system.

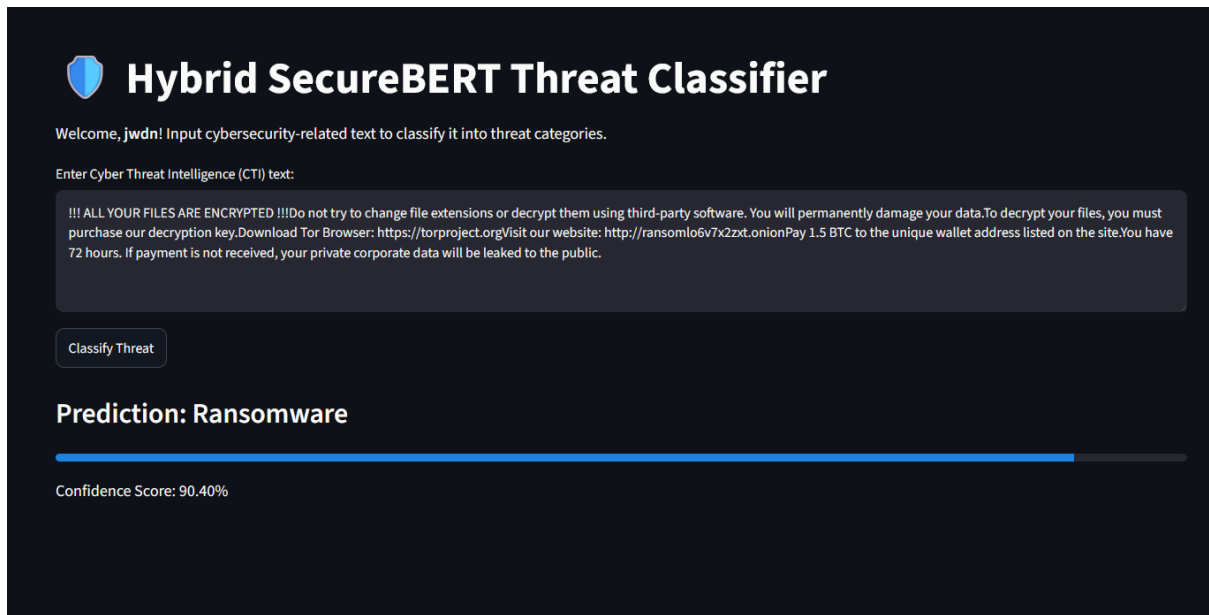


Figure A.3: Prediction result of classified threat

Above Figure illustrates the prediction output generated by the deployed Streamlit dashboard after processing a CTI record. The dashboard displays the predicted threat category, a confidence score, and a visual confidence indicator to improve result interpretation. During inference, the input text passes through the SecureBERT encoder to generate contextual representations, which are subsequently processed by the BiLSTM and Attention layers before the final classification layer produces the predicted threat category. enabling users to classify CTI text through an interactive web application rather than a notebook-based environment